MODUL 15 · TESTING & DEBUG

Debugging, DevTools & Pengujian Otomatis

Memastikan aplikasi stabil sebelum dirilis. Dimulai dari membaca pesan error dan stack trace, debugPrint, breakpoint, Flutter DevTools, dan flutter analyze, lalu menulis unit test, test dengan fake/mock untuk repository Catatan Kuliah, widget test form login, test Provider, golden test, integration test, dan laporan coverage.

Durasi Pertemuan 15 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management dengan Provider
Modul 14: Proyek Catatan Kuliah Bagian 2 (CRUD & Cache Offline)

Capaian Pembelajaran

- Membaca pesan error dan stack trace Flutter serta memperbaiki lima error yang paling umum (overflow, setState setelah dispose, null check, unbounded height, Provider tidak ditemukan).
- Menggunakan debugPrint, developer.log, breakpoint VS Code, dan Flutter DevTools (Inspector, Performance, Network, Memory) untuk menemukan penyebab bug.
- Menjaga kualitas kode dengan flutter analyze, analysis_options.yaml, dan dart fix.
- Menulis unit test dengan test, expect, group, setUp, matcher, test async, dan test exception.
- Mengisolasi pengujian repository dari Supabase dan Hive menggunakan fake class dan mocktail.
- Menulis widget test (pumpWidget, find, tap, enterText, pump, pumpAndSettle) untuk form login dan halaman yang memakai Provider.
- Menjelaskan fungsi golden test dan integration test, serta mengukur coverage dengan flutter test --coverage.

Daftar Isi Modul

1. Membaca Pesan Error & 5 Error Paling Umum
2. debugPrint, log & Breakpoint di VS Code
3. Flutter DevTools: Inspector, Performance, Network, Memory
4. flutter analyze & Lint
5. Unit Test: test, expect, group, setUp & Matcher
6. Unit Test Lanjutan: Async & Exception
7. Fake & Mock: Menguji Repository Tanpa Supabase
8. Widget Test: Interaksi & Validasi Form Login
9. Menguji Provider (NotesProvider)
10. Golden Test, Integration Test & Coverage
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Membaca Pesan Error & 5 Error Paling Umum

Sebelum belajar alat debugging yang canggih, keterampilan paling penting adalah **membaca pesan error**. Hampir semua error Flutter sebenarnya sudah memberi tahu apa masalahnya, di file mana, dan sering kali juga cara memperbaikinya. Sayangnya, banyak mahasiswa langsung menyalin error ke mesin pencari tanpa membacanya.

Di mana error muncul?

- **Terminal / Debug Console VS Code**: pesan lengkap beserta *stack trace*. Ini sumber informasi utama.
- **Layar aplikasi**: pada mode debug, error saat `build()` menampilkan layar merah dengan teks kuning. Error layout menampilkan garis kuning-hitam.
- **Tab Problems VS Code**: error kompilasi (salah tipe, salah nama) yang muncul sebelum aplikasi dijalankan.

Anatomi pesan error Flutter

Berikut contoh pesan asli yang muncul di Debug Console saat `setState()` dipanggil setelah halaman ditutup:

```
══╡ EXCEPTION CAUGHT BY FLUTTER TEST FRAMEWORK ╞═══════════════════
The following assertion was thrown running a test:
setState() called after dispose(): _ProfilSalahState#7b34c(lifecycle state: defunct, not mounted)
This error happens if you call setState() on a State object for a widget that
no longer appears in the widget tree ...
The preferred solution is to cancel the timer or stop listening to the animation
in the dispose() callback. Another solution is to check the "mounted" property of
this object before calling setState() ...

When the exception was thrown, this was the stack:
#0      State.setState.<anonymous closure> (package:flutter/src/widgets/framework.dart:1163:9)
#1      State.setState (package:flutter/src/widgets/framework.dart:1200:6)
#2      _ProfilSalahState._muat (package:catatan_kuliah/demos/error_demos.dart:71:5)
<asynchronous suspension>
```

Baca dengan urutan berikut:

1. **Judul** (`EXCEPTION CAUGHT BY ...`): bagian Flutter mana yang menangkap error, misalnya *WIDGETS LIBRARY* (saat build) atau *RENDERING LIBRARY* (saat layout).
2. **Kalimat pertama**: inti masalahnya. Di sini: `setState() called after dispose()`.
3. **Paragraf penjelasan**: Flutter sering langsung menyarankan solusi. Di sini: cek properti `mounted`.
4. **Stack trace**: daftar pemanggilan fungsi, paling atas adalah yang terakhir dipanggil. **Cari baris pertama yang berasal dari package proyek Anda** (bukan `package:flutter/`). Pada contoh di atas itu baris `#2` : file `error_demos.dart` baris 71 kolom 5.

> 💡 **Tips**
>
> Di Debug Console VS Code, nama file pada stack trace bisa di-klik (`Cmd`/`Ctrl` + klik) dan langsung membuka baris yang bermasalah.

Berikut lima error yang paling sering ditemui mahasiswa, lengkap dengan kode penyebab dan perbaikannya.

Error 1: A RenderFlex overflowed by ... pixels

Artinya: anak-anak `Row` / `Column` butuh ruang lebih besar daripada yang tersedia. Muncul garis kuning-hitam di sisi yang meluap.

```dart
// SALAH: teks panjang di dalam Row tidak punya batas lebar
class KartuMatkulSalah extends StatelessWidget {
  const KartuMatkulSalah({super.key});

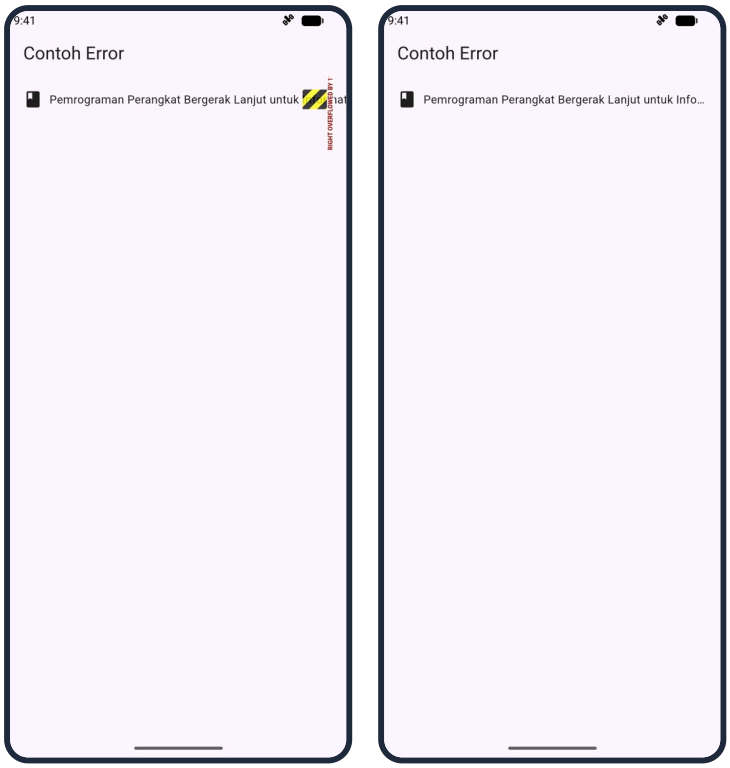
  @override
  Widget build(BuildContext context) {
    return const Row(
      children: [
        Icon(Icons.book),
        SizedBox(width: 8),
        Text('Pemrograman Perangkat Bergerak Lanjut untuk Informatika Semester 5'),
      ],
    );
  }
}
```

Pesan di console: `A RenderFlex overflowed by 173 pixels on the right.` Perbaikannya, bungkus anak yang fleksibel dengan `Expanded` (atau `Flexible`) agar ukurannya mengikuti sisa ruang:

```dart
// BENAR: Expanded memberi Text sisa lebar yang tersedia
class KartuMatkulBenar extends StatelessWidget {
  const KartuMatkulBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return const Row(
      children: [
        Icon(Icons.book),
        SizedBox(width: 8),
        Expanded(
          child: Text(
            'Pemrograman Perangkat Bergerak Lanjut untuk Informatika Semester 5',
            overflow: TextOverflow.ellipsis, // potong dengan "..."
          ),
        ),
      ],
    );
```

```
  }
}
```



Sebelum: teks meluap, muncul garis kuning-hitam di kanan

Sesudah: Expanded + TextOverflow.ellipsis

Untuk `Column` yang terlalu tinggi (misalnya form panjang saat keyboard muncul), solusinya biasanya membungkus `Column` dengan `SingleChildScrollView`.

Error 2: setState() called after dispose()

Artinya: proses async (ambil data, `Future.delayed`, timer) selesai setelah pengguna meninggalkan halaman, lalu kode mencoba memanggil `setState()` pada State yang sudah dibuang.

```dart
class ProfilSalah extends StatefulWidget {
  const ProfilSalah({super.key});

  @override
  State<ProfilSalah> createState() => _ProfilSalahState();
}

class _ProfilSalahState extends State<ProfilSalah> {
  String _nama = 'Memuat...';

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final nama = await ambilProfil(); // butuh 3 detik
```

```
    // Jika pengguna sudah menekan "Back" sebelum 3 detik,
    // State ini sudah di-dispose dan baris berikut memicu error.
    setState(() => _nama = nama);
  }

  @override
  Widget build(BuildContext context) => Text(_nama);
}
```

Perbaikannya: setiap kali ada `await` sebelum `setState()` atau sebelum memakai `context`, cek `mounted` dulu.

```
class ProfilBenar extends StatefulWidget {
  const ProfilBenar({super.key});

  @override
  State<ProfilBenar> createState() => _ProfilBenarState();
}

class _ProfilBenarState extends State<ProfilBenar> {
  String _nama = 'Memuat...';

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final nama = await ambilProfil();
    if (!mounted) return; // widget sudah tidak ada di layar, berhenti
    setState(() => _nama = nama);
  }

  @override
  Widget build(BuildContext context) => Text(_nama);
}
```

Untuk `Timer` dan `StreamSubscription`, batalkan di `dispose()` dengan `timer.cancel()` atau `subscription.cancel()`.

Error 3: Null check operator used on a null value

Artinya: Anda memakai operator `!` (“saya jamin tidak null”) pada nilai yang ternyata null. Ini error paling sering di aplikasi yang membaca data dari API atau database.

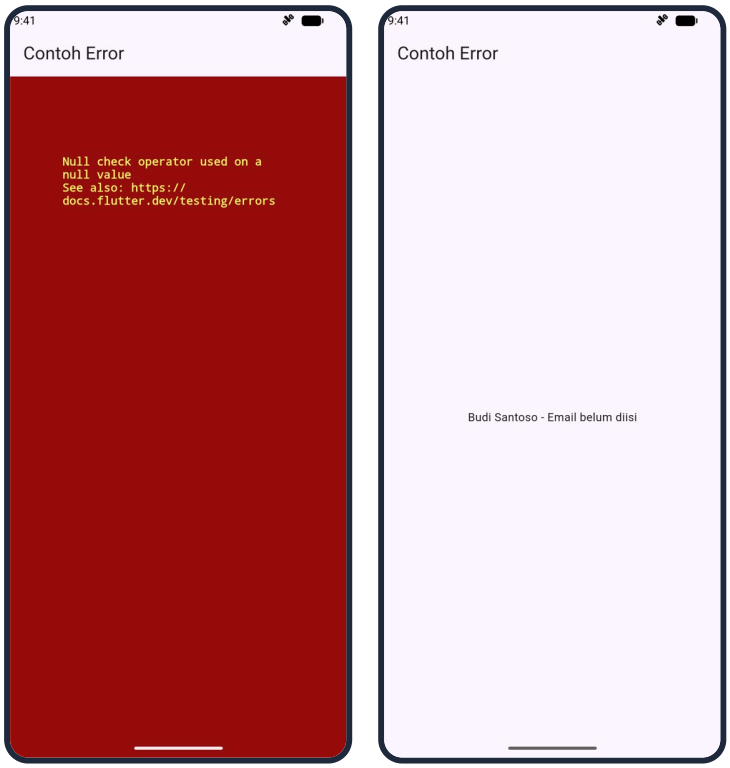
```
class KartuEmail extends StatelessWidget {
  const KartuEmail({super.key, this.mahasiswa});

  final Mahasiswa? mahasiswa;

  @override
  Widget build(BuildContext context) {
    // SALAH: crash jika mahasiswa atau email-nya null
    // return Text(mahasiswa!.email!);
```

```
    // BENAR 1: beri nilai cadangan dengan ?. dan ??
    final email = mahasiswa?.email ?? 'Email belum diisi';

    // BENAR 2: simpan ke variabel lokal, lalu cek null secara eksplisit
    final mhs = mahasiswa;
    if (mhs == null) {
      return const Text('Data mahasiswa tidak ditemukan');
    }
    return Text('${mhs.nama} - $email'); // mhs sudah pasti tidak null
  }
}
```



Versi salah (`mahasiswa!.email!`): layar merah di mode debug

Versi benar: email null diganti teks cadangan

Di build *release*, layar merah ini berubah menjadi area abu-abu kosong. Pengguna tidak melihat pesan apa pun, jadi bug seperti ini harus ditemukan saat debug atau lewat test.

Error 4: Vertical viewport was given unbounded height

Artinya: widget yang bisa di-scroll (`ListView`, `GridView`) diletakkan di dalam widget yang tidak membatasi tingginya (`Column`, atau `ListView` lain). `ListView` ingin setinggi mungkin, `Column` memberi tinggi tak terbatas, sehingga Flutter bingung.

```
// SALAH: ListView di dalam Column -> "Vertical viewport was given unbounded height"
class DaftarSalah extends StatelessWidget {
  const DaftarSalah({super.key});

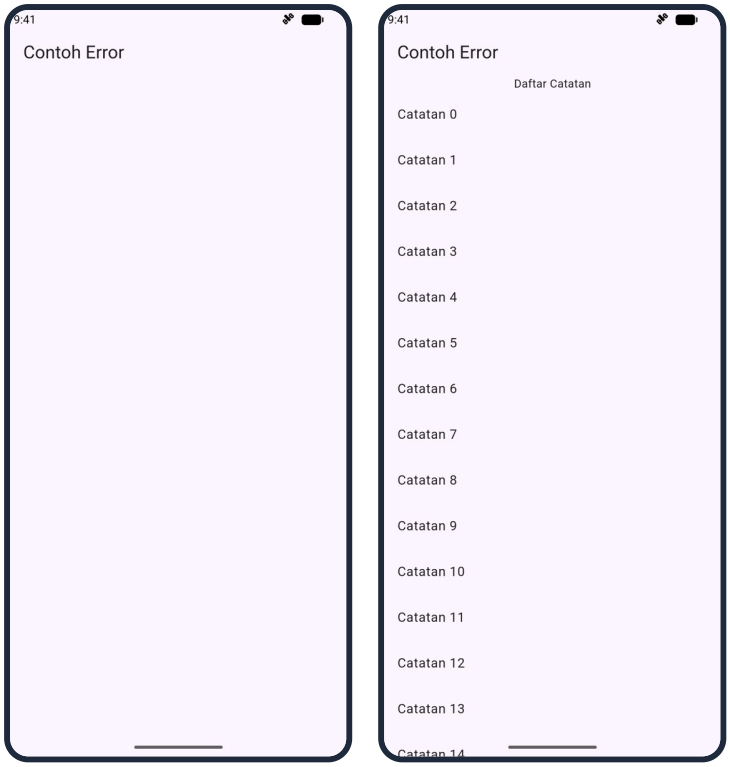
  @override
  Widget build(BuildContext context) {
    return Column(
```

```
      children: [
        const Text('Daftar Catatan'),
        ListView.builder(
          itemCount: 20,
          itemBuilder: (context, i) => ListTile(title: Text('Catatan $i')),
        ),
      ],
    );
  }
}
```

Pesan di console menunjuk langsung ke `ListView` yang bermasalah (`The relevant error-causing widget was: ListView ...error_demos.dart:147:18`). Perbaikannya:

```
// BENAR: Expanded memberi ListView tinggi = sisa ruang Column
class DaftarBenar extends StatelessWidget {
  const DaftarBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        const Text('Daftar Catatan'),
        Expanded(
          child: ListView.builder(
            itemCount: 20,
            itemBuilder: (context, i) => ListTile(title: Text('Catatan $i')),
          ),
        ),
      ],
    );
  }
}
```



Sebelum: halaman kosong, error hanya terlihat di console

Sesudah: ListView dibungkus Expanded

Alternatif lain adalah `ListView(shrinkWrap: true, physics: NeverScrollableScrollPhysics())`, tetapi ini hanya cocok untuk daftar pendek karena semua item dibangun sekaligus.

Error 5: Could not find the correct Provider<T> above this Widget

Artinya: widget memanggil `context.watch<T>()` atau `context.read<T>()`, tetapi tidak ada `Provider<T>` di atasnya pada widget tree. Penyebab paling umum: provider dipasang di dalam satu halaman, lalu halaman lain yang dibuka dengan `Navigator.push` mencoba membacanya.

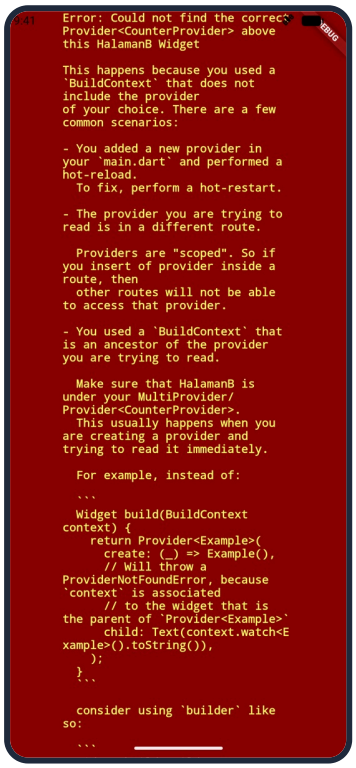
```
// SALAH: provider dipasang DI BAWAH MaterialApp, hanya untuk HalamanA.
// Halaman yang dibuka lewat Navigator.push tidak bisa melihatnya.
class AppSalah extends StatelessWidget {
  const AppSalah({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ChangeNotifierProvider(
        create: (_) => CounterProvider(),
        child: const HalamanA(),
      ),
    );
  }
}
```

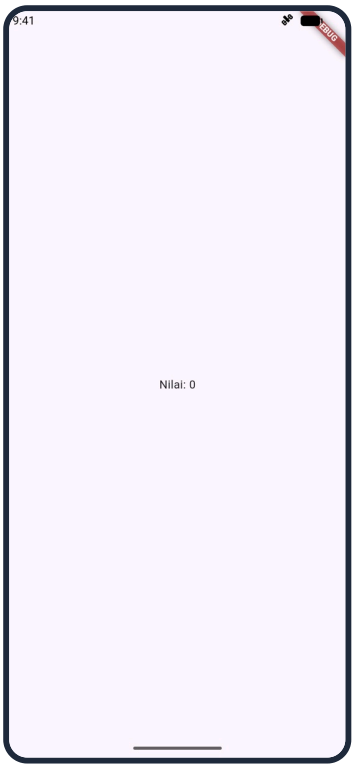
Halaman baru dari `Navigator.push` adalah “saudara” `HalamanA` di bawah `MaterialApp`, bukan anaknya, sehingga tidak bisa melihat provider tersebut. Pindahkan provider ke atas `MaterialApp`:

```dart
// BENAR: provider membungkus MaterialApp sehingga semua route bisa membacanya
class AppBenar extends StatelessWidget {
  const AppBenar({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => CounterProvider(),
      child: const MaterialApp(home: HalamanA()),
    );
  }
}
```



AppSalah: setelah membuka Halaman B, pesan error lengkap dengan daftar penyebab



AppBenar: Halaman B bisa membaca CounterProvider

Penyebab lain: lupa memberi tipe generik (`context.read()` tanpa `<CounterProvider>`), atau menambah provider baru di `main.dart` lalu hanya melakukan *hot reload*. Untuk kasus terakhir, lakukan *hot restart* (`Shift+R` atau tombol restart di VS Code).

Ringkasan

Pesan error	Penyebab umum	Perbaikan
RenderFlex overflowed	Isi Row/Column lebih besar dari layar	`Expanded`, `Flexible`, `SingleChildScrollView`
setState() called after dispose()	setState setelah await, halaman sudah ditutup	`if (!mounted) return;`, cancel timer di dispose
Null check operator used on a null value	Operator `!` pada nilai null	`?.`, `??`, cek null dengan variabel lokal
Vertical viewport was given unbounded height	ListView di dalam Column	Bungkus dengan `Expanded`
Could not find the correct Provider	Provider dipasang terlalu bawah	Pindahkan ke atas `MaterialApp`, hot restart

⚠️ **Kesalahan umum**

- Hanya membaca baris terakhir error, padahal inti masalah ada di kalimat pertama.
- Mencari penyebab di file `framework.dart`. Baris dari `package:flutter/` hampir tidak pernah salah; cari baris pertama dari proyek Anda.
- Menambahkan `!` di mana-mana agar error merah di editor hilang. Ini hanya memindahkan error dari waktu kompilasi ke waktu aplikasi berjalan.
- Memperbaiki overflow dengan mengecilkan font. Di HP lain dengan layar lebih kecil, error akan muncul lagi.

2. debugPrint, log & Breakpoint di VS Code

Setelah tahu letak error, langkah berikutnya adalah memahami *kenapa* kode berperilaku salah. Ada dua cara: mencetak isi variabel ke console (logging), atau menghentikan aplikasi di baris tertentu lalu memeriksa semua variabel (breakpoint).

1. debugPrint: cara paling sederhana

`print()` memang bisa dipakai, tetapi lint bawaan Flutter (`avoid_print`) akan memberi peringatan. Gunakan `debugPrint()` dari `package:flutter/foundation.dart`. Bedanya, `debugPrint` membatasi kecepatan output sehingga log panjang tidak terpotong oleh Android.

```dart
import 'dart:developer' as developer;

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

class KeranjangDebug {
  final List<int> sks = [];

  void tambah(int nilaiSks) {
    debugPrint('tambah() dipanggil dengan sks=$nilaiSks');
    sks.add(nilaiSks);
    debugPrint('isi sekarang: $sks, total=${sks.fold(0, (a, b) => a + b)}');
  }
}
```

Output di Debug Console saat `tambah(3)` lalu `tambah(4)` dipanggil:

```
I/flutter ( 5123): tambah() dipanggil dengan sks=3
I/flutter ( 5123): isi sekarang: [3], total=3
I/flutter ( 5123): tambah() dipanggil dengan sks=4
I/flutter ( 5123): isi sekarang: [3, 4], total=7
```

Tulis log yang **informatif**: sebutkan nama fungsi dan nilai variabelnya, bukan sekadar `debugPrint('masuk')`.

2. log dari dart:developer: log yang lebih terstruktur

`developer.log()` mendukung nama kategori (`name`), objek error, dan stack trace. Log ini tampil rapi di tab *Logging* DevTools dan bisa difilter berdasarkan nama.

```dart
Future<void> muatCatatan() async {
  developer.log('Mulai memuat catatan', name: 'catatan.repo');
  try {
    await Future.delayed(const Duration(seconds: 1));
    throw Exception('Timeout ke server');
  } catch (e, stackTrace) {
    developer.log(
      'Gagal memuat catatan',
      name: 'catatan.repo',
      error: e,
```

```
      stackTrace: stackTrace,
      level: 1000, // 1000 = SEVERE
    );
  }
}
```

Poin penting:

- `catch (e, stackTrace)`: parameter kedua menangkap stack trace, sehingga Anda tahu dari mana error berasal.
- `name: 'catatan.repo'`: kategori log. Di DevTools, ketik `catatan` di kolom filter untuk hanya melihat log ini.
- `level: 1000`: tingkat keparahan (800 = INFO, 900 = WARNING, 1000 = SEVERE).

3. kDebugMode dan assert

Log dan pemeriksaan tambahan sebaiknya hanya aktif saat pengembangan. Flutter menyediakan konstanta `kDebugMode` dan pernyataan `assert`:

```dart
void catatDebug(String pesan) {
  // kDebugMode bernilai false di build release,
  // sehingga blok ini dibuang total oleh compiler.
  if (kDebugMode) {
    debugPrint('[DEBUG] $pesan');
  }
}

double hitungRataRata(List<int> nilai) {
  // assert hanya diperiksa di mode debug
  assert(nilai.isNotEmpty, 'List nilai tidak boleh kosong');
  return nilai.reduce((a, b) => a + b) / nilai.length;
}
```

`assert` sangat berguna untuk menangkap "hal yang seharusnya tidak pernah terjadi". Di mode debug, `hitungRataRata([])` langsung berhenti dengan pesan jelas, bukan menghasilkan `NaN` diam-diam.

4. Breakpoint di VS Code

Breakpoint menghentikan aplikasi tepat di satu baris sehingga Anda bisa melihat nilai *semua* variabel tanpa menulis `debugPrint`. Langkahnya:

1. Jalankan aplikasi dengan mode debug: tekan F5 (atau menu *Run → Start Debugging*). Jangan pakai `flutter run` di terminal jika ingin breakpoint VS Code bekerja.
2. Klik di sebelah kiri nomor baris sampai muncul titik merah.
3. Lakukan aksi di aplikasi yang menjalankan baris tersebut. Aplikasi akan berhenti dan baris tersebut disorot kuning.
4. Lihat panel **Variables** (nilai variabel lokal), **Watch** (ekspresi yang Anda pantau), dan **Call Stack**.

```dart
int hitungTotalSks(List<int> daftarSks) {
  var total = 0;
  for (final sks in daftarSks) {
    total += sks; // <- pasang breakpoint di baris ini (klik kiri nomor baris)
  }
  return total;
}

class TombolHitung extends StatelessWidget {
  const TombolHitung({super.key});

  @override
  Widget build(BuildContext context) {
    return FilledButton(
      onPressed: () {
        final total = hitungTotalSks([3, 4, 2, 3]);
        // Bisa juga berhenti lewat kode (hanya jika debugger terhubung)
        if (total > 10) developer.debugger(message: 'Total SKS > 10');
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Total SKS: $total')),
        );
      },
      child: const Text('Hitung SKS'),
    );
  }
}
```

Saat berhenti di baris `total += sks;`, gunakan tombol kontrol debug:

Tombol	Pintasan	Fungsi
Continue	F5	Lanjut sampai breakpoint berikutnya
Step Over	F10	Jalankan baris ini, berhenti di baris berikutnya
Step Into	F11	Masuk ke dalam fungsi yang dipanggil di baris ini
Step Out	Shift+F11	Selesaikan fungsi saat ini, kembali ke pemanggil
Restart	Shift+Cmd/Ctrl+F5	Hot restart aplikasi

Breakpoint bersyarat dan logpoint

Klik kanan titik merah lalu pilih *Edit Breakpoint*:

- **Expression**: berhenti hanya jika kondisi benar, misalnya `sks > 3`. Berguna di dalam loop panjang.
- **Log Message** (logpoint): tidak berhenti, hanya mencetak pesan seperti `total sekarang {total}`. Ini seperti `debugPrint` tanpa mengubah kode.

Konfigurasi launch.json

Jika aplikasi butuh argumen tambahan (misalnya file konfigurasi Supabase di Modul 16), buat `.vscode/launch.json` lewat menu *Run* → *Add Configuration*:

`.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Catatan Kuliah (debug)",
      "request": "launch",
      "type": "dart",
      "program": "lib/main.dart",
      "args": ["--dart-define-from-file=config/dev.json"]
    },
    {
      "name": "Catatan Kuliah (profile)",
      "request": "launch",
      "type": "dart",
      "flutterMode": "profile",
      "args": ["--dart-define-from-file=config/dev.json"]
    }
  ]
}
```

> ⚠️ **Kesalahan umum**
>
> - Memasang breakpoint lalu menjalankan aplikasi dengan `flutter run` di terminal biasa. Debugger VS Code tidak terhubung, jadi aplikasi tidak pernah berhenti.
> - Meninggalkan puluhan `debugPrint` di kode yang dikumpulkan. Hapus atau bungkus dengan `if (kDebugMode)`.
> - Mencetak data sensitif (password, token Supabase) ke log. Log Android bisa dibaca oleh siapa pun yang memegang HP lewat `adb logcat`.
> - Menulis `catch (e) {}` kosong. Error “tertelan” dan Anda tidak akan pernah tahu ada masalah. Minimal log errornya.

3. Flutter DevTools: Inspector, Performance, Network, Memory

Flutter DevTools adalah kumpulan alat diagnostik berbasis web yang terhubung ke aplikasi yang sedang berjalan. Jika `debugPrint` dan breakpoint adalah "kaca pembesar", DevTools adalah "ruang kontrol" yang menampilkan widget tree, waktu render tiap frame, lalu lintas jaringan, dan pemakaian memori.

Membuka DevTools

```
# Cara 1: dari VS Code
# Jalankan aplikasi dengan F5, lalu Cmd/Ctrl+Shift+P
# ketik "Flutter: Open DevTools" dan pilih tab yang diinginkan

# Cara 2: dari terminal
flutter run
# setelah aplikasi berjalan, terminal menampilkan alamat DevTools, contoh:
# The Flutter DevTools debugger and profiler on sdk gphone64 arm64 is available at:
# http://127.0.0.1:9100?uri=http://127.0.0.1:52341/abc123=/
# Klik/salin alamat tersebut ke browser

# Cara 3: jalankan DevTools mandiri lalu tempel alamat VM service
dart devtools
```

Tab	Dipakai untuk	Mode aplikasi
Inspector	Melihat widget tree, ukuran, padding, constraints	debug
Performance	Mencari frame lambat (jank)	**profile**
CPU Profiler	Fungsi mana yang paling banyak makan waktu CPU	profile
Memory	Pemakaian RAM, mencari memory leak	debug/profile
Network	Request HTTP: URL, status, header, body, durasi	debug/profile
Logging	Output `debugPrint`, `developer.log`, event framework	debug

1. Inspector: memeriksa widget tree dan layout

Inspector menampilkan pohon widget yang *benar-benar* sedang dirender. Fitur yang paling sering dipakai:

- **Select Widget Mode**: klik ikon panah, lalu sentuh elemen di emulator. Inspector melompat ke widget tersebut dan VS Code membuka baris kodenya.
- **Layout Explorer**: untuk `Row` / `Column`, menampilkan ukuran setiap anak, `flex`, dan alignment. Sangat membantu saat memperbaiki RenderFlex overflow.

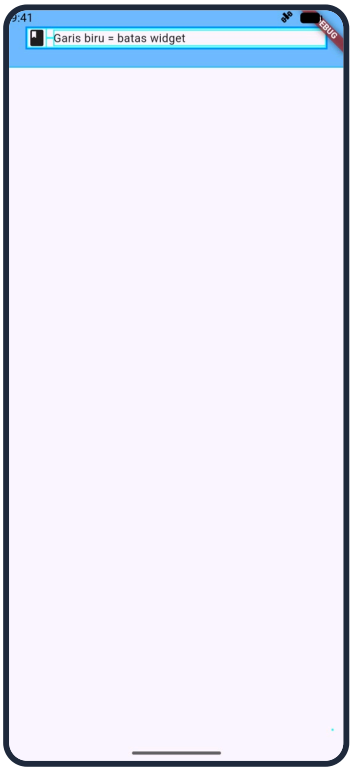
- **Show Guidelines** (debug paint): menggambar garis batas, padding, dan baseline semua widget di layar.

Debug paint juga bisa diaktifkan lewat kode, berguna jika DevTools belum terbuka:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:flutter/rendering.dart';

void main() {
  debugPaintSizeEnabled = true; // tampilkan garis batas semua widget
  runApp(
    const MaterialApp(
      home: Scaffold(
        body: Padding(
          padding: EdgeInsets.all(24),
          child: Row(
            children: [
              Icon(Icons.book),
              SizedBox(width: 8),
              Expanded(child: Text('Garis biru = batas widget')),
            ],
          ),
        ),
      ),
    ),
  );
}
```



debugPaintSizeEnabled: setiap widget digambar dengan garis batas, padding, dan arah layout

Jangan lupa menghapus baris `debugPaintSizeEnabled = true` setelah selesai.

2. Performance: mencari jank

Layar 60 Hz menggambar 60 frame per detik, artinya setiap frame punya jatah sekitar **16 ms**. Jika satu frame butuh lebih lama, animasi terlihat patah-patah (*jank*). Penyebab paling umum: pekerjaan berat di dalam `build()`.

```
// SALAH: perhitungan berat di dalam build() -> frame macet (jank)
int hitungPrimaSampai(int batas) {
  var jumlah = 0;
  for (var n = 2; n <= batas; n++) {
    var prima = true;
    for (var d = 2; d * d <= n; d++) {
      if (n % d == 0) {
        prima = false;
        break;
      }
    }
    if (prima) jumlah++;
  }
  return jumlah;
}

class StatistikSalah extends StatelessWidget {
  const StatistikSalah({super.key});

  @override
  Widget build(BuildContext context) {
    final jumlah = hitungPrimaSampai(5000000); // dihitung ulang tiap rebuild!
    return Text('Ada $jumlah bilangan prima');
  }
}
```

Di tab Performance, frame-frame yang lambat tampil sebagai batang merah. Klik batang tersebut untuk melihat bahwa `hitungPrimaSampai` menghabiskan ratusan milidetik di thread UI. Perbaikannya, pindahkan perhitungan ke isolate lain dengan `Isolate.run` dan simpan Future-nya di `initState`:

```
// BENAR: jalankan di isolate lain, hasilnya ditampilkan lewat FutureBuilder
class StatistikBenar extends StatefulWidget {
  const StatistikBenar({super.key});

  @override
  State<StatistikBenar> createState() => _StatistikBenarState();
}

class _StatistikBenarState extends State<StatistikBenar> {
  // Future dibuat SEKALI di initState, bukan di build()
  late final Future<int> _hasil;

  @override
  void initState() {
    super.initState();
    _hasil = Isolate.run(() => hitungPrimaSampai(5000000));
```

```
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<int>(
      future: _hasil,
      builder: (context, snapshot) {
        if (!snapshot.hasData) return const CircularProgressIndicator();
        return Text('Ada ${snapshot.data} bilangan prima');
      },
    );
  }
}
```

Ukur performa dengan mode **profile**, bukan debug. Mode debug sengaja lambat karena banyak pemeriksaan tambahan, sehingga hasilnya menyesatkan.

```
# Mode profile hanya berjalan di HP fisik atau emulator, tidak di web debug
flutter run --profile
```

Sebagai alternatif cepat tanpa DevTools, aktifkan grafik performa langsung di atas aplikasi:

```
class AppDenganOverlay extends StatelessWidget {
  const AppDenganOverlay({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      // Menampilkan grafik waktu frame (UI & raster) di atas aplikasi
      showPerformanceOverlay: true,
      home: Scaffold(body: Center(child: Text('Cek grafik di atas'))),
    );
  }
}
```



Performance overlay: grafik atas = thread raster (GPU), grafik bawah = thread UI. Garis horizontal menandai kelipatan 16 ms; penanda frame terbaru hijau jika di bawah batas dan merah jika melewatinya.

3. Network: memeriksa request HTTP

Tab Network mencatat setiap request yang dibuat dengan `package:http`, `dio`, maupun client Supabase. Untuk tiap request Anda bisa melihat:

- Method dan URL lengkap (apakah query `?select=*` sudah benar?).
- Status code: `200` sukses, `401` belum login / token kedaluwarsa, `404` URL salah, `500` error server.
- Header request (apakah header `Authorization` terkirim?) dan body response (JSON yang benar-benar dikirim server).
- Durasi: request yang butuh 3 detik akan terasa lambat bagi pengguna.

Saat Catatan Kuliah gagal memuat data, buka tab ini *sebelum* menebak-nebak. Jika status `200` tetapi daftar tetap kosong, masalahnya ada di parsing JSON atau RLS, bukan koneksi.

4. Memory: mencari memory leak

Memory leak terjadi jika objek yang sudah tidak dipakai tetap tersimpan di RAM. Di Flutter, penyebab paling umum adalah controller yang tidak di-`dispose()`:

```
class HalamanCari extends StatefulWidget {
  const HalamanCari({super.key});

  @override
  State<HalamanCari> createState() => _HalamanCariState();
```

```dart
}

class _HalamanCariState extends State<HalamanCari> {
  final _controller = TextEditingController();
  final _scrollController = ScrollController();

  @override
  void dispose() {
    // Tanpa dua baris ini, controller tetap tinggal di memori
    // setiap kali halaman dibuka-tutup (memory leak).
    _controller.dispose();
    _scrollController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: TextField(controller: _controller)),
      body: ListView.builder(
        controller: _scrollController,
        itemCount: 100,
        itemBuilder: (context, i) => ListTile(title: Text('Hasil $i')),
      ),
    );
  }
}
```

Cara memeriksanya di tab Memory:

1. Buka tab Memory, klik *Take heap snapshot* (snapshot A).
2. Buka lalu tutup `HalamanCari` sebanyak 10 kali.
3. Klik *GC* (paksa garbage collection), lalu ambil snapshot B.
4. Bandingkan (*Diff*) snapshot B dengan A dan cari `TextEditingController`. Jika jumlahnya bertambah 10, ada leak.

Objek yang wajib di-dispose: `TextEditingController`, `ScrollController`, `AnimationController`, `FocusNode`, `TabController`, `StreamSubscription` (cancel), dan `Timer` (cancel).

⚠️ **Kesalahan umum**

- Mengukur performa di mode debug lalu menyimpulkan “Flutter lambat”. Selalu ukur dengan `flutter run --profile`.
- Membuat `Future` langsung di parameter `FutureBuilder(future: ambilData())`. Setiap rebuild memicu request baru; simpan Future di `initState`.
- Lupa memanggil `super.dispose()` di akhir method `dispose()`.
- Membiarkan `showPerformanceOverlay: true` atau `debugPaintSizeEnabled = true` sampai build rilis.

4. flutter analyze & Lint

Bug yang paling murah adalah bug yang tertangkap *sebelum* aplikasi dijalankan. Dart menyediakan **analyzer** yang membaca kode Anda dan melaporkan error, peringatan, dan saran gaya penulisan (lint). Biasakan menjalankan `flutter analyze` sebelum commit dan sebelum mengumpulkan tugas.

Menjalankan flutter analyze

```
# Dari folder root proyek
flutter analyze

# Hanya folder tertentu
flutter analyze lib/pages
```

Misalkan ada kode "gaya lama" seperti ini:

```dart
import 'package:flutter/material.dart';

class Contoh extends StatelessWidget {
  Contoh({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    var judul = 'Catatan';
    print(judul);
    return Container(
      child: Text(judul),
    );
  }
}
```

Hasil `flutter analyze` (output asli):

```
info • Constructors in '@immutable' classes should be declared as 'const' • lib/contoh.dart:4:3 •
prefer_const_constructors_in_immutables
info • Parameter 'key' could be a super parameter • lib/contoh.dart:4:3 • use_super_parameters
info • Don't invoke 'print' in production code. Try using a logging framework • lib/contoh.dart:9:5
• avoid_print
info • Unnecessary instance of 'Container' • lib/contoh.dart:10:12 • avoid_unnecessary_containers

4 issues found.
```

Setiap baris berisi: **tingkat** (`error`, `warning`, `info`), **pesan**, **lokasi** (file:baris:kolom), dan **nama aturan**. Nama aturan bisa dicari di dart.dev/tools/linter-rules untuk penjelasan lengkap dan contoh.

Tingkat	Artinya	Wajib diperbaiki?
`error`	Kode tidak bisa dikompilasi (salah tipe, nama tidak ada)	Ya, aplikasi tidak akan jalan
`warning`	Kode jalan tetapi hampir pasti salah (variabel tidak dipakai, import tidak terpakai)	Ya
`info`	Saran gaya dan praktik terbaik dari lint	Sangat disarankan; target tugas: 0 issue

Memperbaiki otomatis dengan dart fix

Banyak lint bisa diperbaiki otomatis. Lihat dulu apa yang akan diubah, baru terapkan:

```
# Pratinjau perubahan
dart fix --dry-run

# Terapkan semua perbaikan
dart fix --apply

# Rapikan format (indentasi, spasi, koma)
dart format lib test
```

Pada contoh di atas, `dart fix --apply` mengubah konstruktor menjadi `const Contoh({super.key});`. Lint `avoid_print` dan `avoid_unnecessary_containers` harus Anda perbaiki sendiri (ganti dengan `debugPrint` dan hapus `Container`).

Mengatur aturan di analysis_options.yaml

Proyek baru dari `flutter create` sudah memakai paket `flutter_lints`. Anda bisa menambah aturan yang lebih ketat atau menaikkan tingkat suatu aturan:

`analysis_options.yaml`

```yaml
include: package:flutter_lints/flutter.yaml

analyzer:
  errors:
    # Naikkan avoid_print dari info menjadi error
    avoid_print: error
  exclude:
    - "**/*.g.dart" # file hasil generator (misalnya adapter Hive)
    - build/**
    - android/**
    - ios/**
    - web/**
    - windows/**
    - macos/**
    - linux/**

linter:
  rules:
```

```
    prefer_single_quotes: true      # pakai 'kutip satu'
    prefer_final_locals: true       # variabel lokal yang tidak diubah wajib final
    always_declare_return_types: true
    unawaited_futures: true         # Future yang lupa di-await akan ditandai
```

Aturan `unawaited_futures` sangat berguna untuk aplikasi Supabase/Hive: ia menandai pemanggilan seperti `repository.addNote(...)` yang lupa diberi `await`, sehingga error-nya tidak tertangkap oleh `try/catch`.

Mengabaikan lint secara sengaja

Kadang ada alasan kuat untuk melanggar aturan. Abaikan hanya di baris itu dan tulis alasannya:

```dart
// Skrip seeding data hanya dijalankan developer dari terminal
// ignore: avoid_print
void cetakRingkasan(int jumlah) => print('Berhasil membuat $jumlah catatan');
```

Jangan menulis `// ignore_for_file:` di atas file hanya agar daftar issue terlihat bersih.

> ⚠️ **Kesalahan umum**
>
> - Mengabaikan semua `info` karena “cuma saran”. Lint seperti `use_build_context_synchronously` menandai bug sungguhan (memakai `context` setelah `await` tanpa cek `mounted`).
> - Menghapus baris `include: package:flutter_lints/flutter.yaml` agar warning hilang.
> - Menjalankan `dart fix --apply` tanpa commit terlebih dahulu. Jika hasilnya tidak sesuai, Anda tidak bisa kembali dengan mudah.

5. Unit Test: test, expect, group, setUp & Matcher

Debugging mencari bug yang *sudah* terjadi. Testing mencegah bug yang *akan* terjadi. Dengan test otomatis, setiap kali Anda mengubah kode, cukup jalankan satu perintah untuk memastikan fitur lama tidak rusak (regresi).

Piramida pengujian Flutter

Jenis	Yang diuji	Kecepatan	Butuh emulator?	Jumlah ideal
Unit test	Satu fungsi/class: validator, perhitungan, repository, provider	Milidetik	Tidak	Paling banyak
Widget test	Satu halaman/widget: tampilan dan interaksi	Ratusan ms	Tidak	Sedang
Integration test	Aplikasi utuh di perangkat sungguhan	Menit	Ya	Sedikit, untuk alur penting

Struktur folder test

Semua test berada di folder `test/` dan nama filenya **wajib** diakhiri `_test.dart`. Biasakan struktur folder test meniru folder `lib/`:

```
catatan_kuliah/
├── lib/
|   ├── utils/ipk.dart
|   ├── utils/validators.dart
|   └── models/keranjang_krs.dart
└── test/
    ├── unit/ipk_test.dart
    ├── unit/validators_test.dart
    └── unit/keranjang_krs_test.dart
```

Package `flutter_test` sudah otomatis ada di `dev_dependencies` proyek baru, jadi tidak perlu dipasang.

Test pertama: fungsi hitungIpk

Agar mudah diuji, letakkan logika di file terpisah di `lib/`, bukan di dalam widget:

`lib/utils/ipk.dart`

```dart
// lib/utils/ipk.dart
double hitungIpk(List<int> bobot, List<int> sks) {
  if (bobot.length != sks.length) {
    throw ArgumentError('Jumlah bobot dan SKS harus sama');
```

```
  }
  if (bobot.isEmpty) return 0.0;

  var totalBobot = 0;
  var totalSks = 0;
  for (var i = 0; i < bobot.length; i++) {
    totalBobot += bobot[i] * sks[i];
    totalSks += sks[i];
  }
  return double.parse((totalBobot / totalSks).toStringAsFixed(2));
}

String predikat(double ipk) {
  if (ipk >= 3.51) return 'Dengan Pujian';
  if (ipk >= 3.01) return 'Sangat Memuaskan';
  if (ipk >= 2.76) return 'Memuaskan';
  return 'Cukup';
}
```

Lalu tulis test-nya. Ganti `catatan_kuliah` pada import dengan nama proyek Anda (baris `name:` di `pubspec.yaml`).

`test/unit/ipk_test.dart`

```
// test/unit/ipk_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/utils/ipk.dart';

void main() {
  group('hitungIpk', () {
    test('menghitung IPK normal dengan benar', () {
      // Arrange: siapkan data
      final bobot = [4, 4, 3, 4]; // A, A, B, A
      final sks = [3, 3, 3, 3];

      // Act: jalankan fungsi yang diuji
      final hasil = hitungIpk(bobot, sks);

      // Assert: periksa hasilnya
      expect(hasil, 3.75);
    });

    test('daftar kosong menghasilkan 0.0', () {
      expect(hitungIpk([], []), 0.0);
    });

    test('SKS berbeda ikut menjadi bobot', () {
      // (4 x 4 + 2 x 2) / 6 = 3.33
      expect(hitungIpk([4, 2], [4, 2]), 3.33);
    });
  });

  group('predikat', () {
    test('IPK 3.75 berpredikat Dengan Pujian', () {
      expect(predikat(3.75), 'Dengan Pujian');
    });
```

```
    test('batas bawah tiap predikat', () {
      expect(predikat(3.51), 'Dengan Pujian');
      expect(predikat(3.01), 'Sangat Memuaskan');
      expect(predikat(2.76), 'Memuaskan');
      expect(predikat(2.0), 'Cukup');
    });
  });
}
```

Penjelasan:

- `test('deskripsi', () { ... })` mendefinisikan satu kasus uji. Tulis deskripsi sebagai kalimat yang menjelaskan perilaku, karena kalimat inilah yang muncul saat test gagal.
- `expect(nilaiAktual, nilaiHarapan)` adalah inti test. Jika keduanya berbeda, test gagal.
- `group()` mengelompokkan test yang berkaitan. Nama grup ditambahkan di depan nama test pada laporan.
- Pola **Arrange-Act-Assert**: siapkan data, jalankan fungsi, periksa hasil. Pola ini membuat test mudah dibaca.

Menjalankan test

```
# Semua test di folder test/
flutter test

# Satu file saja
flutter test test/unit/ipk_test.dart

# Hanya test yang namanya mengandung kata tertentu
flutter test --plain-name "predikat"

# Output lebih rinci (satu baris per test)
flutter test --reporter expanded
```

Contoh output saat semua lulus dan saat ada yang gagal:

```
# Semua lulus
00:01 +5: All tests passed!

# Ada yang gagal (expect(hasil, 3.8) padahal hasilnya 3.75)
00:00 +0 -1: hitungIpk menghitung IPK normal dengan benar [E]
  Expected: <3.8>
    Actual: <3.75>

  package:matcher                                  expect
  package:flutter_test/src/widget_tester.dart 473:18  expect
  test/unit/ipk_test.dart 16:7                     main.<fn>.<fn>
...
00:00 +4 -1: Some tests failed.
```

`+4 -1` berarti 4 lulus, 1 gagal. Laporan menunjukkan nilai yang diharapkan, nilai aktual, dan baris test yang gagal. Di VS Code, Anda juga bisa menekan tombol ▶ *Run* yang muncul di atas setiap

`test()` atau membuka panel *Testing* (ikon tabung kimia).

setUp dan tearDown

Jika beberapa test membutuhkan objek yang sama, jangan membuatnya sekali di luar lalu dipakai bersama, karena perubahan di satu test akan bocor ke test lain. Gunakan `setUp()` yang dijalankan ulang **sebelum setiap test**.

lib/models/keranjang_krs.dart

```dart
// lib/models/keranjang_krs.dart
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;

  const MataKuliah(this.kode, this.nama, this.sks);
}

class KeranjangKrs {
  static const batasSks = 24;
  final List<MataKuliah> _items = [];

  List<MataKuliah> get items => List.unmodifiable(_items);
  int get totalSks => _items.fold(0, (total, mk) => total + mk.sks);

  void tambah(MataKuliah mk) {
    if (_items.any((item) => item.kode == mk.kode)) {
      throw StateError('${mk.nama} sudah ada di KRS');
    }
    if (totalSks + mk.sks > batasSks) {
      throw StateError('Total SKS melebihi $batasSks');
    }
    _items.add(mk);
  }

  void hapus(String kode) => _items.removeWhere((mk) => mk.kode == kode);

  void kosongkan() => _items.clear();
}
```

test/unit/keranjang_krs_test.dart

```dart
// test/unit/keranjang_krs_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/models/keranjang_krs.dart';

void main() {
  late KeranjangKrs keranjang;

  const mobile = MataKuliah('IF301', 'Pemrograman Mobile', 3);
  const basdat = MataKuliah('IF205', 'Basis Data', 4);

  // Dijalankan SEBELUM setiap test: keranjang selalu baru dan kosong
  setUp(() {
    keranjang = KeranjangKrs();
```

```
  });

  test('keranjang baru kosong', () {
    expect(keranjang.items, isEmpty);
    expect(keranjang.totalSks, 0);
  });

  test('menambah mata kuliah menambah total SKS', () {
    keranjang.tambah(mobile);
    keranjang.tambah(basdat);

    expect(keranjang.items, hasLength(2));
    expect(keranjang.totalSks, 7);
  });

  test('menghapus mata kuliah berdasarkan kode', () {
    keranjang.tambah(mobile);
    keranjang.hapus('IF301');

    expect(keranjang.items, isEmpty);
  });

  test('daftar items tidak bisa diubah dari luar', () {
    expect(() => keranjang.items.add(mobile), throwsUnsupportedError);
  });
}
```

- `late KeranjangKrs keranjang;` dideklarasikan di luar, lalu diisi ulang di `setUp`. Test kedua tidak terpengaruh mata kuliah yang ditambahkan di test lain.
- `tearDown()` adalah kebalikannya: dijalankan setelah setiap test, misalnya untuk menutup box Hive atau menghapus file sementara.
- `setUpAll()` / `tearDownAll()` dijalankan sekali saja untuk seluruh file, cocok untuk pekerjaan mahal seperti membuka database tes.

Matcher: cara membandingkan yang lebih ekspresif

Argumen kedua `expect` bisa berupa nilai biasa atau **matcher**. Matcher membuat test lebih mudah dibaca dan pesan gagalnya lebih jelas.

`test/unit/matchers_test.dart`

```
// test/unit/matchers_test.dart
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('contoh matcher yang sering dipakai', () {
    final nilai = [80, 90, 75];
    const nama = 'Pemrograman Mobile';

    expect(nilai.length, equals(3)); // sama dengan 3
    expect(nilai, contains(90)); // list berisi 90
    expect(nilai, hasLength(3)); // panjang list 3
    expect(nilai, isNot(contains(100))); // TIDAK berisi 100
    expect(nama, startsWith('Pemrograman'));
    expect(nama, contains('Mobile'));
```

```
    expect(nama, isA<String>()); // bertipe String
    expect(0.1 + 0.2, closeTo(0.3, 0.0001)); // untuk angka desimal
    expect(85, greaterThan(80));
    expect(85, inInclusiveRange(80, 100));
    expect(<String>[], isEmpty);
    expect(nilai, everyElement(greaterThanOrEqualTo(75)));
    expect(nilai, orderedEquals([80, 90, 75]));
    expect(nilai, unorderedEquals([75, 80, 90]));
  });
}
```

Matcher	Lulus jika
`equals(x)` atau langsung `x`	Nilai sama dengan `x` (list dibandingkan isinya)
`isTrue`, `isFalse`, `isNull`, `isNotNull`	Sesuai namanya
`isEmpty`, `isNotEmpty`, `hasLength(n)`	Untuk String, List, Map
`contains(x)`	String memuat teks / List memuat elemen
`isA<T>()`	Objek bertipe `T`
`closeTo(x, delta)`	Angka desimal mendekati `x`
`greaterThan`, `lessThan`, `inInclusiveRange`	Perbandingan angka
`isNot(m)`	Kebalikan matcher `m`
`throwsA(m)`, `throwsArgumentError`, `throwsStateError`	Fungsi melempar error (bagian berikutnya)

Contoh nyata: menguji validator form login

Validator yang dipakai form login di Modul 13 sebaiknya diletakkan di file sendiri agar bisa diuji tanpa UI:

`lib/utils/validators.dart`

```
// lib/utils/validators.dart
final _polaEmail = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');

String? validasiEmail(String? value) {
  if (value == null || value.trim().isEmpty) return 'Email wajib diisi';
  if (!_polaEmail.hasMatch(value.trim())) return 'Format email tidak valid';
  return null;
}
```

```dart
String? validasiPassword(String? value) {
  if (value == null || value.isEmpty) return 'Password wajib diisi';
  if (value.length < 8) return 'Password minimal 8 karakter';
  return null;
}
```

test/unit/validators_test.dart

```dart
// test/unit/validators_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/utils/validators.dart';

void main() {
  group('validasiEmail', () {
    test('email kosong ditolak', () {
      expect(validasiEmail(''), 'Email wajib diisi');
      expect(validasiEmail(null), 'Email wajib diisi');
      expect(validasiEmail('   '), 'Email wajib diisi');
    });

    test('format salah ditolak', () {
      expect(validasiEmail('budi'), 'Format email tidak valid');
      expect(validasiEmail('budi@kampus'), 'Format email tidak valid');
    });

    test('email benar lolos (mengembalikan null)', () {
      expect(validasiEmail('budi@mhs.unsoed.ac.id'), isNull);
    });
  });

  group('validasiPassword', () {
    test('kurang dari 8 karakter ditolak', () {
      expect(validasiPassword('abc123'), contains('minimal 8'));
    });

    test('8 karakter atau lebih lolos', () {
      expect(validasiPassword('rahasia123'), isNull);
    });
  });
}
```

Perhatikan bahwa setiap aturan diuji dari dua sisi: **kasus positif** (input benar lolos) dan **kasus negatif** (input salah ditolak). Juga uji **kasus batas**: string berisi spasi saja, nilai `null`, panjang tepat 8 karakter.

⚠️ **Kesalahan umum**

- Nama file tidak diakhiri `_test.dart`, sehingga `flutter test` tidak menemukannya.
- Import memakai path relatif ke `lib/` (`import '../../lib/utils/ipk.dart'`). Selalu gunakan `package:nama_proyek/...`.
- Membandingkan `double` dengan `==`: `0.1 + 0.2` tidak sama persis dengan `0.3`. Gunakan `closeTo`.
- Satu test menguji terlalu banyak hal. Jika gagal, Anda sulit tahu bagian mana yang salah. Lebih baik banyak test kecil.
- Test saling bergantung (test B hanya lulus jika test A dijalankan dulu). Gunakan `setUp` agar setiap test mulai dari kondisi bersih.

6. Unit Test Lanjutan: Async & Exception

Kode aplikasi nyata penuh dengan `Future` (ambil data dari Supabase, baca Hive) dan error (input tidak valid, koneksi putus). Bagian ini membahas cara menguji keduanya.

1. Menguji Future

Cara paling mudah: jadikan fungsi test `async`, lalu `await` hasilnya seperti di kode biasa. Alternatifnya, gunakan matcher `completion()`.

`test/unit/async_test.dart`

```dart
// test/unit/async_test.dart
import 'package:flutter_test/flutter_test.dart';

Future<String> ambilNamaMahasiswa(String nim) async {
  await Future.delayed(const Duration(milliseconds: 100));
  if (nim.isEmpty) throw ArgumentError('NIM kosong');
  return 'Budi Santoso';
}

Stream<int> hitungMundur(int dari) async* {
  for (var i = dari; i >= 0; i--) {
    yield i;
  }
}

void main() {
  test('cara 1: async/await di dalam test', () async {
    final nama = await ambilNamaMahasiswa('H1D021001');
    expect(nama, 'Budi Santoso');
  });

  test('cara 2: matcher completion', () {
    expect(ambilNamaMahasiswa('H1D021001'), completion('Budi Santoso'));
  });

  test('Future yang gagal diuji dengan expectLater + throwsA', () async {
    await expectLater(
      ambilNamaMahasiswa(''),
      throwsA(isA<ArgumentError>()),
    );
  });

  test('Stream mengeluarkan nilai berurutan', () {
    expect(hitungMundur(3), emitsInOrder([3, 2, 1, 0, emitsDone]));
  });
}
```

Penjelasan:

- **Cara 1** (`async` / `await`) paling mudah dibaca dan paling sering dipakai.
- **Cara 2** `completion(x)`: test menunggu Future selesai lalu membandingkan hasilnya dengan `x`.

- Untuk Future yang **gagal**, gunakan `await expectLater(future, throwsA(...))`. Kata `await` penting: tanpa itu test bisa selesai sebelum Future-nya gagal.
- `emitsInOrder` menguji Stream: nilai apa saja yang keluar dan urutannya. `emitsDone` memastikan Stream sudah ditutup.

2. Menguji exception pada fungsi biasa

Test juga harus memastikan kode *menolak* input yang salah. Kuncinya: bungkus pemanggilan dengan `() =>` agar `expect` yang menjalankan fungsinya dan menangkap error.

`test/unit/exception_test.dart`

```dart
// test/unit/exception_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/models/keranjang_krs.dart';
import 'package:catatan_kuliah/utils/ipk.dart';

void main() {
  test('hitungIpk melempar ArgumentError jika panjang list beda', () {
    // Bungkus pemanggilan dengan () => supaya error ditangkap oleh expect
    expect(() => hitungIpk([4, 3], [3]), throwsArgumentError);
  });

  test('menambah matkul yang sama dua kali melempar StateError', () {
    final keranjang = KeranjangKrs();
    const mk = MataKuliah('IF301', 'Pemrograman Mobile', 3);
    keranjang.tambah(mk);

    expect(() => keranjang.tambah(mk), throwsStateError);
  });

  test('pesan error melebihi 24 SKS bisa diperiksa', () {
    final keranjang = KeranjangKrs();
    for (var i = 0; i < 6; i++) {
      keranjang.tambah(MataKuliah('MK$i', 'Matkul $i', 4)); // 24 SKS
    }

    expect(
      () => keranjang.tambah(const MataKuliah('MK9', 'Matkul 9', 2)),
      throwsA(
        isA<StateError>().having(
          (e) => e.message,
          'message',
          contains('melebihi 24'),
        ),
      ),
    );
  });
}
```

- `throwsArgumentError`, `throwsStateError`, `throwsUnsupportedError`, `throwsException` adalah matcher siap pakai untuk tipe error umum.
- `throwsA(isA<StateError>())` adalah bentuk umumnya, bisa dipakai untuk class exception buatan sendiri, misalnya `throwsA(isA<AuthException>())`.

- `.having((e) => e.message, 'message', contains(...))` memeriksa isi properti error. Berguna jika satu fungsi bisa melempar beberapa pesan berbeda.

Perbedaan kecil yang sering membuat bingung

```
// SALAH: hitungIpk dijalankan SEBELUM expect, error tidak tertangkap
// dan test gagal dengan pesan "ArgumentError" mentah.
// expect(hitungIpk([4], []), throwsArgumentError);

// BENAR: expect yang menjalankan fungsi di dalam closure
// expect(() => hitungIpk([4], []), throwsArgumentError);

// BENAR untuk fungsi async: future langsung, tanpa closure, dan di-await
// await expectLater(repository.addNote('', 'isi'), throwsArgumentError);
```

3. Ringkasan pola test async dan error

Yang diuji	Pola
Future berhasil	`expect(await f(), hasil)`
Future gagal	`await expectLater(f(), throwsA(isA<E>()))`
Fungsi biasa melempar error	`expect(() => f(), throwsA(isA<E>()))`
Stream	`expect(stream, emitsInOrder([a, b, emitsDone]))`
Fungsi TIDAK melempar error	`expect(() => f(), returnsNormally)`

> ⚠️ **Kesalahan umum**
>
> - Lupa `async` pada fungsi test padahal di dalamnya ada `await` (error kompilasi), atau lupa `await` pada `expectLater` (test lulus palsu).
> - Memakai `Future.delayed` berdetik-detik di kode yang diuji. Test jadi lambat. Pada widget test, waktu bisa “dipercepat” dengan `tester.pump(Duration)`.
> - Menguji fungsi yang langsung memanggil Supabase. Test menjadi lambat, butuh internet, dan mengotori database. Solusinya ada di bagian berikutnya: *fake* dan *mock*.

7. Fake & Mock: Menguji Repository Tanpa Supabase

Di Modul 14, `NoteRepository` mengambil catatan dari Supabase dan menyimpannya ke cache Hive. Bagaimana mengujinya tanpa internet, tanpa akun Supabase, dan tanpa mengotori database? Jawabannya: ganti sumber data asli dengan **pengganti palsu** yang kita kendalikan penuh.

> ℹ️ **Versi ringkas, bukan salinan kode proyek**
>
> Agar setiap contoh di modul ini bisa dijalankan sendiri, class `Note`, `NoteRepository`, dan data source di bawah sengaja dibuat lebih sederhana daripada kode proyek Modul 14 (nama method dan field bisa berbeda). Konsepnya sama persis. Test untuk kode proyek yang sesungguhnya ada di Modul 14 section *Unit Test* (`test/note_repository_test.dart` dan `test/note_local_data_source_test.dart`).

Kenapa butuh pengganti?

- **Cepat**: tidak ada request jaringan, ratusan test selesai dalam hitungan detik.
- **Stabil**: hasil test tidak bergantung pada koneksi atau isi database hari ini.
- **Bisa mensimulasikan kegagalan**: kondisi "offline" sulit dibuat dengan server sungguhan, tetapi mudah dengan pengganti palsu.

Istilah	Arti	Contoh
Fake	Implementasi sederhana yang benar-benar berfungsi, ditulis tangan	Data disimpan di `List`, bukan di Supabase
Mock	Objek yang perilakunya diatur per test dan bisa diperiksa pemanggilannya	`when(...).thenAnswer(...)`, `verify(...)` dengan mocktail

Langkah 1: kode harus bergantung pada abstraksi

Pengganti hanya bisa dipasang jika repository tidak membuat koneksi Supabase/Hive sendiri, melainkan **menerimanya dari luar** lewat konstruktor (*dependency injection*). Berikut versi sederhana arsitektur Modul 14:

📄 **`lib/models/note.dart`**

```dart
// lib/models/note.dart
class Note {
  final String id;
  final String title;
```

```
  final String content;

  const Note({required this.id, required this.title, required this.content});
}
```

lib/data/note_data_source.dart

```
// lib/data/note_data_source.dart
import '../models/note.dart';

/// Sumber data online (di Modul 14: tabel notes di Supabase).
abstract class NoteRemoteDataSource {
  Future<List<Note>> fetchNotes();
  Future<Note> createNote(String title, String content);
}

/// Cache offline (di Modul 14: box Hive).
abstract class NoteLocalCache {
  Future<void> saveNotes(List<Note> notes);
  Future<List<Note>> readNotes();
}
```

lib/data/note_repository.dart

```
// lib/data/note_repository.dart
import '../models/note.dart';
import 'note_data_source.dart';

class NoteRepository {
  NoteRepository({required this.remote, required this.cache});

  final NoteRemoteDataSource remote;
  final NoteLocalCache cache;

  /// true jika data terakhir diambil dari cache (sedang offline).
  bool lastFromCache = false;

  Future<List<Note>> getNotes() async {
    try {
      final notes = await remote.fetchNotes();
      await cache.saveNotes(notes);
      lastFromCache = false;
      return notes;
    } catch (_) {
      lastFromCache = true;
      return cache.readNotes();
    }
  }

  Future<Note> addNote(String title, String content) async {
    if (title.trim().isEmpty) {
      throw ArgumentError('Judul tidak boleh kosong');
    }
    return remote.createNote(title.trim(), content);
  }
}
```

`NoteRepository` hanya mengenal `NoteRemoteDataSource` dan `NoteLocalCache` (abstract class). Di aplikasi asli, kita memasang implementasi Supabase dan Hive. Di test, kita memasang versi palsu.

Langkah 2: menulis fake class

Fake ditulis dengan `implements`, sehingga compiler memaksa kita mengisi semua method. Tambahkan "tombol" untuk mengatur perilaku (`shouldFail`) dan "penghitung" untuk diperiksa (`fetchCount`).

`test/mock/fakes.dart`

```dart
// test/mock/fakes.dart
import 'package:catatan_kuliah/data/note_data_source.dart';
import 'package:catatan_kuliah/models/note.dart';

/// Pengganti Supabase: datanya disimpan di list biasa.
class FakeRemoteDataSource implements NoteRemoteDataSource {
  FakeRemoteDataSource({List<Note>? awal}) : notes = [...?awal];

  final List<Note> notes;
  bool shouldFail = false; // ubah ke true untuk mensimulasikan offline
  int fetchCount = 0; // berapa kali fetchNotes dipanggil

  @override
  Future<List<Note>> fetchNotes() async {
    fetchCount++;
    if (shouldFail) throw Exception('Tidak ada koneksi internet');
    return List.of(notes);
  }

  @override
  Future<Note> createNote(String title, String content) async {
    if (shouldFail) throw Exception('Tidak ada koneksi internet');
    final note = Note(id: 'n${notes.length + 1}', title: title, content: content);
    notes.add(note);
    return note;
  }
}

/// Pengganti Hive: cache disimpan di memori.
class FakeLocalCache implements NoteLocalCache {
  List<Note> saved = [];

  @override
  Future<void> saveNotes(List<Note> notes) async => saved = List.of(notes);

  @override
  Future<List<Note>> readNotes() async => List.of(saved);
}
```

File ini diletakkan di folder `test/` karena hanya dipakai untuk pengujian. Namanya tidak diakhiri `_test.dart`, jadi tidak dijalankan sebagai test.

Langkah 3: menguji repository dengan fake

`test/mock/note_repository_test.dart`

```dart
// test/mock/note_repository_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';

import 'fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late FakeLocalCache cache;
  late NoteRepository repository;

  const catatan1 = Note(id: 'n1', title: 'Pertemuan 1', content: 'Pengenalan Dart');

  setUp(() {
    remote = FakeRemoteDataSource(awal: [catatan1]);
    cache = FakeLocalCache();
    repository = NoteRepository(remote: remote, cache: cache);
  });

  test('online: data diambil dari remote lalu disimpan ke cache', () async {
    final notes = await repository.getNotes();

    expect(notes, hasLength(1));
    expect(notes.first.title, 'Pertemuan 1');
    expect(cache.saved, hasLength(1)); // cache ikut terisi
    expect(repository.lastFromCache, isFalse);
  });

  test('offline: data diambil dari cache', () async {
    cache.saved = [catatan1];
    remote.shouldFail = true;

    final notes = await repository.getNotes();

    expect(notes.single.id, 'n1');
    expect(repository.lastFromCache, isTrue);
  });

  test('offline dan cache kosong: hasilnya list kosong, bukan crash', () async {
    remote.shouldFail = true;

    expect(await repository.getNotes(), isEmpty);
  });

  test('addNote menolak judul kosong tanpa memanggil remote', () async {
    await expectLater(repository.addNote('   ', 'isi'), throwsArgumentError);
    expect(remote.notes, hasLength(1)); // tidak bertambah
  });

  test('addNote merapikan spasi judul', () async {
    final note = await repository.addNote('  Kuis 1  ', 'Bab 1-3');

    expect(note.title, 'Kuis 1');
```

```
    expect(remote.notes, hasLength(2));
  });
}
```

Lima test ini mendokumentasikan perilaku penting repository: online menyimpan cache, offline membaca cache, offline tanpa cache tidak crash, dan validasi judul. Jika suatu hari ada yang mengubah `getNotes()` dan lupa menyimpan cache, test pertama langsung gagal.

Alternatif: mocktail

Menulis fake butuh waktu jika interface-nya besar. Package `mocktail` membuat mock tanpa menulis method satu per satu. Pasang sebagai *dev dependency*:

```
flutter pub add dev:mocktail
```

`test/mock/note_repository_mocktail_test.dart`

```
// test/mock/note_repository_mocktail_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';
import 'package:catatan_kuliah/data/note_data_source.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';

// Cukup satu baris per class, tanpa menulis method apa pun
class MockRemote extends Mock implements NoteRemoteDataSource {}

class MockCache extends Mock implements NoteLocalCache {}

void main() {
  late MockRemote remote;
  late MockCache cache;
  late NoteRepository repository;

  const catatan = Note(id: 'n1', title: 'UTS', content: 'Bab 1-6');

  setUp(() {
    remote = MockRemote();
    cache = MockCache();
    repository = NoteRepository(remote: remote, cache: cache);
  });

  test('online: hasil remote disimpan ke cache', () async {
    // Atur perilaku mock (stub)
    when(() => remote.fetchNotes()).thenAnswer((_) async => [catatan]);
    when(() => cache.saveNotes(any())).thenAnswer((_) async {});

    final notes = await repository.getNotes();

    expect(notes, [catatan]);
    // Pastikan saveNotes dipanggil tepat satu kali dengan data yang benar
    verify(() => cache.saveNotes([catatan])).called(1);
    verifyNever(() => cache.readNotes());
  });
```

```
  test('offline: remote melempar error, cache yang dibaca', () async {
    when(() => remote.fetchNotes()).thenThrow(Exception('offline'));
    when(() => cache.readNotes()).thenAnswer((_) async => [catatan]);

    final notes = await repository.getNotes();

    expect(notes, [catatan]);
    expect(repository.lastFromCache, isTrue);
    verifyNever(() => cache.saveNotes(any()));
  });
}
```

API mocktail	Fungsi
`class MockX extends Mock implements X {}`	Membuat class mock
`when(() => mock.f()).thenReturn(nilai)`	Method biasa mengembalikan nilai
`when(() => mock.f()).thenAnswer((_) async => nilai)`	Method yang mengembalikan Future
`when(() => mock.f()).thenThrow(error)`	Method melempar error
`any()`	Argumen apa saja
`verify(() => mock.f()).called(n)`	Memastikan method dipanggil n kali
`verifyNever(() => mock.f())`	Memastikan method tidak pernah dipanggil

Jika memakai `any()` untuk argumen bertipe class buatan sendiri (misalnya `Note`), mocktail meminta Anda mendaftarkan nilai cadangan sekali di `setUpAll(() => registerFallbackValue(const Note(id: '', title: '', content: '')));`. Untuk `List<Note>` seperti contoh di atas, hal ini tidak diperlukan.

Fake atau mock?

- Pakai **fake** jika pengganti yang sama dipakai di banyak file test (repository, provider, widget test). Contoh di modul ini memakai fake yang sama di tiga tempat.
- Pakai **mock** jika Anda ingin memeriksa *interaksi*: method apa yang dipanggil, berapa kali, dengan argumen apa.

⚠️ **Kesalahan umum**

- Membuat koneksi di dalam class (`final client = Supabase.instance.client;` di dalam repository). Class seperti ini tidak bisa diuji tanpa Supabase sungguhan. Terima dependensi lewat konstruktor.
- Lupa melakukan stub: memanggil method mock yang belum diatur dengan `when` menghasilkan error `type 'Null' is not a subtype of type 'Future<...>'`.
- Memakai `thenReturn` untuk method yang mengembalikan Future. Gunakan `thenAnswer((_) async => ...)`.
- Hanya menguji jalur sukses. Bagian paling berharga dari fake/mock justru kemampuannya mensimulasikan kegagalan.

8. Widget Test: Interaksi & Validasi Form Login

Widget test menjalankan widget di lingkungan virtual (tanpa emulator), lalu mensimulasikan interaksi pengguna: mengetik, menekan tombol, menggulir. Hasilnya diperiksa dengan cara yang sama seperti unit test, yaitu `expect`.

Kosakata dasar WidgetTester

Perintah	Fungsi
`testWidgets('...', (tester) async { })`	Mendefinisikan widget test; `tester` adalah "robot" yang mengendalikan UI
`await tester.pumpWidget(widget)`	Merender widget untuk pertama kali
`find.text('...')`, `find.byKey(...)`, `find.byType(...)`, `find.byIcon(...)`	Mencari widget di layar (finder)
`await tester.tap(finder)`	Menekan widget
`await tester.enterText(finder, 'teks')`	Mengetik ke `TextField` / `TextFormField`
`await tester.pump()`	Menggambar satu frame berikutnya (setelah setState)
`await tester.pump(Duration)`	Memajukan waktu virtual lalu menggambar frame
`await tester.pumpAndSettle()`	Terus menggambar frame sampai tidak ada animasi/timer yang tersisa
`findsOneWidget`, `findsNothing`, `findsNWidgets(n)`, `findsWidgets`	Matcher jumlah widget yang ditemukan

Contoh 1: aplikasi penghitung klik

Kita mulai dari aplikasi paling sederhana. Simpan sebagai `lib/main.dart`:

`lib/main.dart`

```dart
// lib/main.dart
import 'package:flutter/material.dart';

void main() {
  runApp(const KlikApp());
}
```

```dart
class KlikApp extends StatelessWidget {
  const KlikApp({super.key});

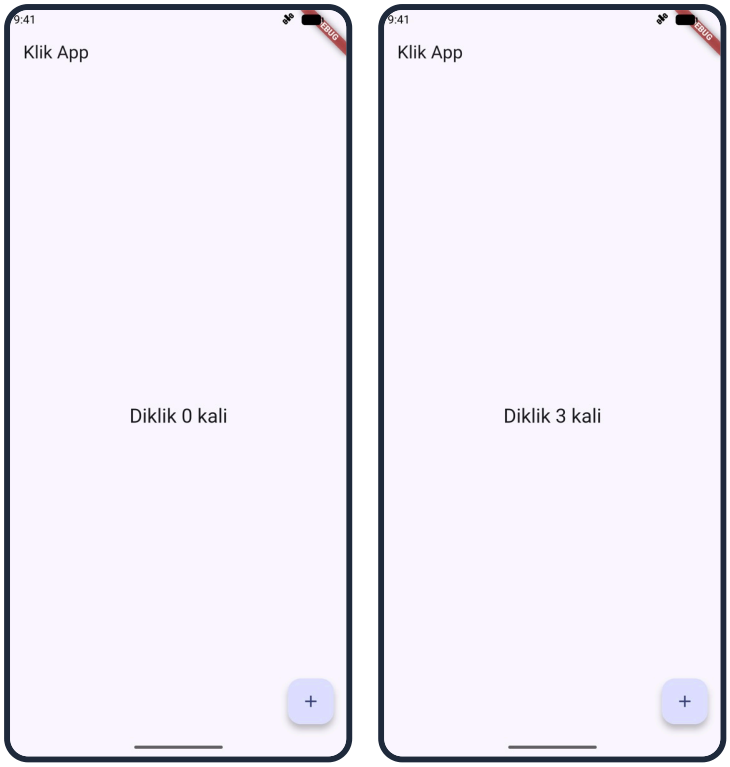
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Klik App',
      theme: ThemeData(colorSchemeSeed: Colors.indigo),
      home: const KlikPage(),
    );
  }
}

class KlikPage extends StatefulWidget {
  const KlikPage({super.key});

  @override
  State<KlikPage> createState() => _KlikPageState();
}

class _KlikPageState extends State<KlikPage> {
  int _jumlah = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Klik App')),
      body: Center(
        child: Text(
          'Diklik $_jumlah kali',
          style: Theme.of(context).textTheme.headlineSmall,
        ),
      ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah',
        onPressed: () => setState(() => _jumlah++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Kondisi awal: "Diklik 0 kali" Setelah tombol + ditekan tiga kali

Test berikut melakukan hal yang sama dengan yang Anda lakukan secara manual di emulator, tetapi otomatis:

test/widget/klik_app_test.dart

```
// test/widget/klik_app_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/main.dart';

void main() {
  testWidgets('angka bertambah saat tombol + ditekan', (tester) async {
    // 1. Render widget
    await tester.pumpWidget(const KlikApp());

    // 2. Periksa kondisi awal
    expect(find.text('Diklik 0 kali'), findsOneWidget);
    expect(find.text('Diklik 1 kali'), findsNothing);

    // 3. Tekan tombol, lalu minta Flutter menggambar frame baru
    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();

    // 4. Periksa hasilnya
    expect(find.text('Diklik 1 kali'), findsOneWidget);
  });

  testWidgets('menekan tombol tiga kali', (tester) async {
    await tester.pumpWidget(const KlikApp());

    for (var i = 0; i < 3; i++) {
      await tester.tap(find.byTooltip('Tambah'));
    }
```

```
    await tester.pump();

    expect(find.text('Diklik 3 kali'), findsOneWidget);
  });
}
```

Poin penting:

- `tester.tap()` hanya mengirim sentuhan. Layar **belum** berubah sampai Anda memanggil `pump()`. Ini kesalahan nomor satu pemula.
- `find.byTooltip('Tambah')` mencari tombol lewat tooltip-nya. Ini sekaligus memastikan tombol punya label aksesibilitas.
- Test dimulai dengan memeriksa kondisi awal. Tanpa itu, test bisa lulus walaupun teks awal sudah salah.

Contoh 2: berbagai cara mencari widget (finder)

test/widget/finder_test.dart

```
// test/widget/finder_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  testWidgets('berbagai cara mencari widget', (tester) async {
    await tester.pumpWidget(
      MaterialApp(
        home: Scaffold(
          appBar: AppBar(title: const Text('Profil')),
          body: Column(
            children: [
              const Text('Budi Santoso', key: Key('nama')),
              const Icon(Icons.verified),
              ElevatedButton(onPressed: () {}, child: const Text('Edit')),
              const Text('Edit'),
            ],
          ),
        ),
      ),
    );

    expect(find.text('Profil'), findsOneWidget); // berdasarkan teks
    expect(find.byKey(const Key('nama')), findsOneWidget); // berdasarkan Key
    expect(find.byIcon(Icons.verified), findsOneWidget); // berdasarkan ikon
    expect(find.byType(ElevatedButton), findsOneWidget); // berdasarkan tipe
    expect(find.text('Edit'), findsNWidgets(2)); // ada 2 teks "Edit"
    // Teks "Edit" yang berada DI DALAM ElevatedButton saja
    expect(find.widgetWithText(ElevatedButton, 'Edit'), findsOneWidget);
    expect(find.text('Hapus'), findsNothing);

    // Membaca properti widget yang ditemukan
    final teksNama = tester.widget<Text>(find.byKey(const Key('nama')));
    expect(teksNama.data, 'Budi Santoso');
```

```
  });
}
```

Urutan prioritas memilih finder: `find.byKey` paling stabil (tidak terpengaruh perubahan teks), `find.text` paling mudah dibaca, `find.byType` jika hanya ada satu widget sejenis. `tester.widget<T>(finder)` mengambil objek widget-nya untuk memeriksa properti.

pump() vs pumpAndSettle()

	`pump()`	`pumpAndSettle()`
Jumlah frame	Tepat satu	Sebanyak yang dibutuhkan sampai tenang
Cocok untuk	setState sederhana, memeriksa kondisi "di tengah" (loading)	Navigasi, dialog, SnackBar, Future yang sudah selesai
Bahaya	Animasi belum selesai saat diperiksa	**Timeout** jika ada animasi tanpa akhir (misalnya `CircularProgressIndicator` yang terus berputar)

Contoh 3: menguji validasi form login (Modul 13)

Form login Catatan Kuliah adalah kandidat ideal untuk widget test: banyak aturan validasi, dan bug di sini membuat pengguna tidak bisa masuk sama sekali. Agar mudah diuji, proses login **tidak** dipanggil langsung di dalam halaman, melainkan diterima lewat parameter `onLogin`. Di aplikasi asli isinya `context.read<AuthProvider>().signIn`; di test isinya fungsi palsu.

`lib/pages/login_page.dart`

```dart
// lib/pages/login_page.dart
import 'package:flutter/material.dart';

import '../utils/validators.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key, required this.onLogin});

  /// Dipanggil saat form valid. Di Modul 13 isinya AuthProvider.signIn.
  final Future<void> Function(String email, String password) onLogin;

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _loading = false;
```

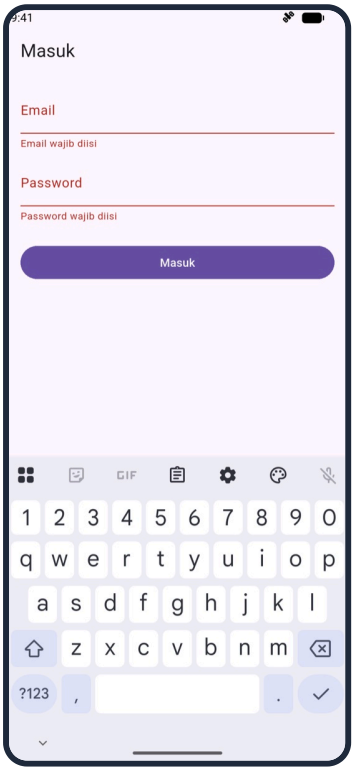
```dart
@override
void dispose() {
  _emailController.dispose();
  _passwordController.dispose();
  super.dispose();
}

Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;

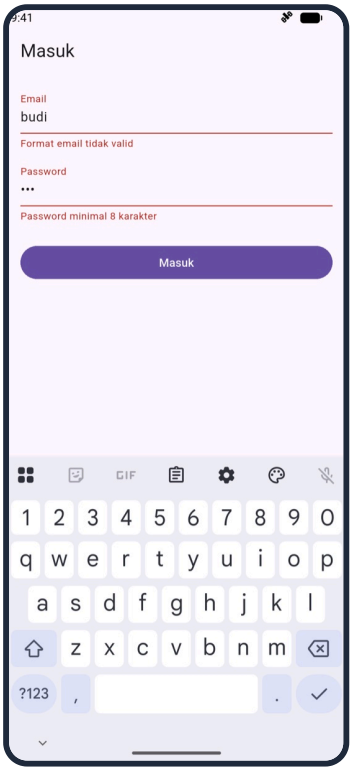
  setState(() => _loading = true);
  try {
    await widget.onLogin(
      _emailController.text.trim(),
      _passwordController.text,
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Login gagal, periksa email/password')),
    );
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Masuk')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Form(
        key: _formKey,
        child: Column(
          children: [
            TextFormField(
              key: const Key('field-email'),
              controller: _emailController,
              decoration: const InputDecoration(labelText: 'Email'),
              keyboardType: TextInputType.emailAddress,
              validator: validasiEmail,
            ),
            const SizedBox(height: 12),
            TextFormField(
              key: const Key('field-password'),
              controller: _passwordController,
              decoration: const InputDecoration(labelText: 'Password'),
              obscureText: true,
              validator: validasiPassword,
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              child: FilledButton(
                onPressed: _loading ? null : _submit,
                child: _loading
                    ? const SizedBox(
```

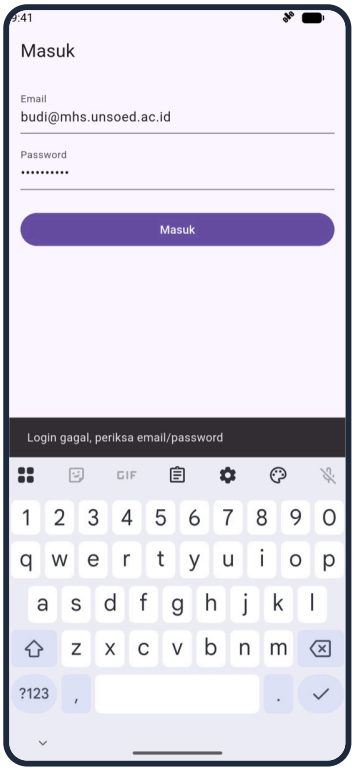
```
                        width: 20,
                        height: 20,
                        child: CircularProgressIndicator(strokeWidth: 2),
                      )
                    : const Text('Masuk'),
              ),
            ),
          ],
        ),
      ),
    ),
  );
  }
}
```



Tombol Masuk ditekan saat form kosong



Email "budi" dan password "123"



Form valid tetapi onLogin melempar error: SnackBar muncul

Ketiga kondisi pada screenshot di atas kini kita uji secara otomatis, ditambah dua skenario lain:

test/widget/login_page_test.dart

```
// test/widget/login_page_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/pages/login_page.dart';

void main() {
  // Helper agar setiap test tidak menulis MaterialApp berulang kali
  Future<void> pumpLogin(
    WidgetTester tester, {
    Future<void> Function(String, String)? onLogin,
  }) async {
    await tester.pumpWidget(
```

```
    MaterialApp(
      home: LoginPage(onLogin: onLogin ?? (email, password) async {}),
    ),
  );
}

final fieldEmail = find.byKey(const Key('field-email'));
final fieldPassword = find.byKey(const Key('field-password'));
final tombolMasuk = find.widgetWithText(FilledButton, 'Masuk');

testWidgets('form kosong menampilkan dua pesan error', (tester) async {
  await pumpLogin(tester);

  await tester.tap(tombolMasuk);
  await tester.pump();

  expect(find.text('Email wajib diisi'), findsOneWidget);
  expect(find.text('Password wajib diisi'), findsOneWidget);
});

testWidgets('email salah format dan password pendek', (tester) async {
  await pumpLogin(tester);

  await tester.enterText(fieldEmail, 'budi');
  await tester.enterText(fieldPassword, '123');
  await tester.tap(tombolMasuk);
  await tester.pump();

  expect(find.text('Format email tidak valid'), findsOneWidget);
  expect(find.text('Password minimal 8 karakter'), findsOneWidget);
});

testWidgets('form valid memanggil onLogin dengan data yang diketik',
    (tester) async {
  String? emailDikirim;
  String? passwordDikirim;

  await pumpLogin(
    tester,
    onLogin: (email, password) async {
      emailDikirim = email;
      passwordDikirim = password;
    },
  );

  await tester.enterText(fieldEmail, '  budi@mhs.unsoed.ac.id ');
  await tester.enterText(fieldPassword, 'rahasia123');
  await tester.tap(tombolMasuk);
  await tester.pumpAndSettle();

  expect(emailDikirim, 'budi@mhs.unsoed.ac.id'); // sudah di-trim
  expect(passwordDikirim, 'rahasia123');
  expect(find.text('Email wajib diisi'), findsNothing);
});

testWidgets('login gagal menampilkan SnackBar', (tester) async {
  await pumpLogin(
    tester,
    onLogin: (email, password) async => throw Exception('401'),
```

```
    );

    await tester.enterText(fieldEmail, 'budi@mhs.unsoed.ac.id');
    await tester.enterText(fieldPassword, 'salahsalah');
    await tester.tap(tombolMasuk);
    await tester.pumpAndSettle();

    expect(find.text('Login gagal, periksa email/password'), findsOneWidget);
  });

  testWidgets('tombol menampilkan loading selama proses login',
      (tester) async {
    await pumpLogin(
      tester,
      onLogin: (email, password) =>
          Future.delayed(const Duration(seconds: 2)),
    );

    await tester.enterText(fieldEmail, 'budi@mhs.unsoed.ac.id');
    await tester.enterText(fieldPassword, 'rahasia123');
    await tester.tap(tombolMasuk);
    await tester.pump(); // satu frame: setState(_loading = true) terlihat

    expect(find.byType(CircularProgressIndicator), findsOneWidget);

    await tester.pump(const Duration(seconds: 2)); // majukan waktu 2 detik
    await tester.pump(); // frame setelah loading selesai

    expect(find.byType(CircularProgressIndicator), findsNothing);
  });
}
```

Penjelasan bagian penting:

- **Helper `pumpLogin`**: membungkus `LoginPage` dengan `MaterialApp` (wajib, karena `Scaffold`, `TextFormField`, dan `SnackBar` butuh tema dan `ScaffoldMessenger`).
- **`Key('field-email')`**: diberikan di `LoginPage` agar test bisa menemukan field dengan pasti. Mencari lewat `find.text('Email')` rawan karena label bisa berubah.
- **`find.widgetWithText(FilledButton, 'Masuk')`**: teks “Masuk” juga ada di judul AppBar, jadi kita batasi pencarian ke tombol saja.
- **Menangkap argumen**: variabel `emailDikirim` diisi oleh fungsi palsu, lalu diperiksa. Ini cara sederhana memastikan form mengirim data yang benar (termasuk `trim()`).
- **Test loading**: `pump()` sekali untuk melihat kondisi tengah (indikator loading tampil), lalu `pump(Duration(seconds: 2))` memajukan waktu virtual. Test tetap selesai dalam milidetik walaupun “menunggu” 2 detik. Di sini `pumpAndSettle()` tidak bisa dipakai di tengah, karena indikator loading berputar terus.

Menggulir sampai widget terlihat

Jika widget berada di bawah layar dalam `ListView`, ia mungkin belum dibangun sehingga `find` tidak menemukannya. Gulir dulu:

```dart
// di dalam testWidgets, setelah pumpWidget sebuah ListView berisi 100 item
await tester.scrollUntilVisible(
  find.text('Catatan 80'),
  300, // jarak gulir per langkah (pixel)
  scrollable: find.byType(Scrollable).first,
);
expect(find.text('Catatan 80'), findsOneWidget);
```

⚠ Kesalahan umum

- Lupa `await` di depan `tester.tap`, `tester.pump`, atau `tester.enterText`.
- Lupa memanggil `pump()` setelah `tap()`, lalu bingung kenapa teks belum berubah.
- Memanggil `pumpAndSettle()` saat ada animasi tak berujung, sehingga test gagal dengan pesan `pumpAndSettle timed out`.
- Me-render halaman tanpa `MaterialApp`, menghasilkan error `No MediaQuery widget ancestor found` atau `No Material widget found`.
- Halaman langsung memanggil `Supabase.instance.client`. Di test, Supabase belum diinisialisasi sehingga test crash. Pisahkan aksi lewat parameter atau provider (bagian berikutnya).

9. Menguji Provider (NotesProvider)

Di Modul 14, logika halaman catatan dipindahkan ke `NotesProvider`. Kabar baiknya: `ChangeNotifier` adalah class Dart biasa, sehingga bisa diuji dengan **unit test** tanpa widget sama sekali. Setelah itu, kita uji halaman yang memakai provider tersebut dengan **widget test**.

Provider yang akan diuji

`lib/providers/notes_provider.dart`

```dart
// lib/providers/notes_provider.dart
import 'package:flutter/foundation.dart';

import '../data/note_repository.dart';
import '../models/note.dart';

enum NotesStatus { initial, loading, loaded, error }

class NotesProvider extends ChangeNotifier {
  NotesProvider(this._repository);

  final NoteRepository _repository;

  NotesStatus status = NotesStatus.initial;
  List<Note> notes = [];
  String? errorMessage;
  bool get isOffline => _repository.lastFromCache;

  Future<void> loadNotes() async {
    status = NotesStatus.loading;
    notifyListeners();
    try {
      notes = await _repository.getNotes();
      status = NotesStatus.loaded;
    } catch (e) {
      errorMessage = 'Gagal memuat catatan';
      status = NotesStatus.error;
    }
    notifyListeners();
  }

  Future<bool> addNote(String title, String content) async {
    try {
      final note = await _repository.addNote(title, content);
      notes = [note, ...notes];
      notifyListeners();
      return true;
    } catch (e) {
      errorMessage = 'Gagal menyimpan catatan';
      notifyListeners();
      return false;
    }
  }
}
```

Langkah 1: unit test untuk ChangeNotifier

Provider dibuat dengan repository sungguhan yang diberi data source palsu dari bagian sebelumnya (`test/mock/fakes.dart`). Dengan begitu kita sekaligus menguji kerja sama provider dan repository.

`test/unit/notes_provider_test.dart`

```dart
// test/unit/notes_provider_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:catatan_kuliah/providers/notes_provider.dart';

import '../mock/fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late NotesProvider provider;

  setUp(() {
    remote = FakeRemoteDataSource(
      awal: [const Note(id: 'n1', title: 'Pertemuan 1', content: 'Dart')],
    );
    final repository = NoteRepository(remote: remote, cache: FakeLocalCache());
    provider = NotesProvider(repository);
  });

  test('status awal initial dan list kosong', () {
    expect(provider.status, NotesStatus.initial);
    expect(provider.notes, isEmpty);
  });

  test('loadNotes: loading lalu loaded', () async {
    final riwayatStatus = <NotesStatus>[];
    provider.addListener(() => riwayatStatus.add(provider.status));

    await provider.loadNotes();

    expect(riwayatStatus, [NotesStatus.loading, NotesStatus.loaded]);
    expect(provider.notes, hasLength(1));
  });

  test('addNote menaruh catatan baru di urutan teratas', () async {
    await provider.loadNotes();

    final berhasil = await provider.addNote('Kuis', 'Bab 2');

    expect(berhasil, isTrue);
    expect(provider.notes.first.title, 'Kuis');
    expect(provider.notes, hasLength(2));
  });

  test('addNote gagal saat offline, errorMessage terisi', () async {
    remote.shouldFail = true;

    final berhasil = await provider.addNote('Kuis', 'Bab 2');
```

```
    expect(berhasil, isFalse);
    expect(provider.errorMessage, 'Gagal menyimpan catatan');
  });

  test('provider memanggil notifyListeners saat addNote', () async {
    var jumlahNotifikasi = 0;
    provider.addListener(() => jumlahNotifikasi++);

    await provider.addNote('Kuis', 'Bab 2');

    expect(jumlahNotifikasi, 1);
  });
}
```

Teknik yang perlu diperhatikan:

- `provider.addListener(...)`: inilah cara widget "mendengar" provider. Di test, kita memakainya untuk merekam urutan status (`loading` lalu `loaded`) dan menghitung berapa kali `notifyListeners()` dipanggil.
- **Menguji jumlah notifikasi** mencegah dua bug yang sering terjadi: lupa memanggil `notifyListeners()` (UI tidak berubah) dan memanggilnya berlebihan (UI rebuild tanpa perlu).
- `remote.shouldFail = true`: mensimulasikan kondisi offline cukup dengan satu baris.

Langkah 2: halaman yang memakai provider

`lib/pages/notes_page.dart`

```
// lib/pages/notes_page.dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

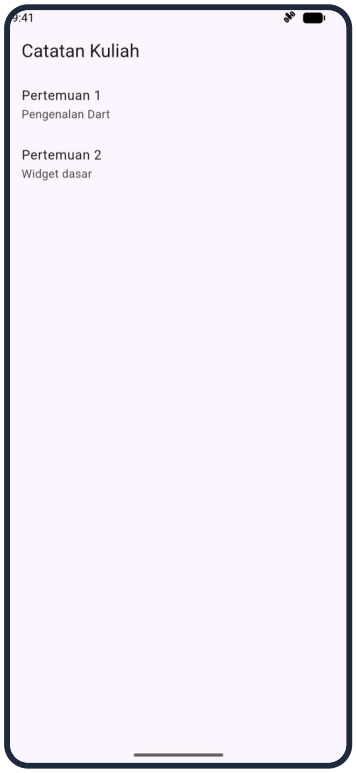
import '../providers/notes_provider.dart';

class NotesPage extends StatelessWidget {
  const NotesPage({super.key});

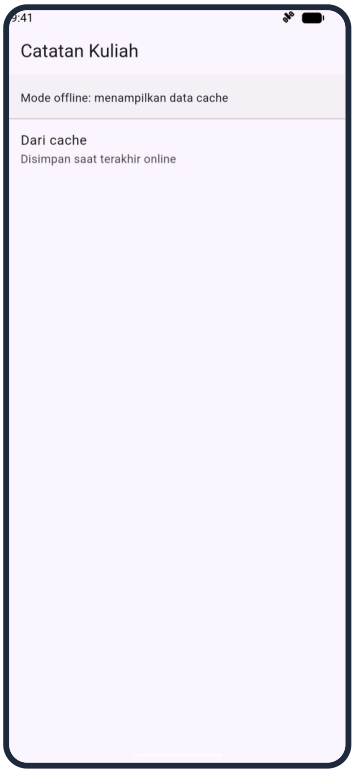
  @override
  Widget build(BuildContext context) {
    final provider = context.watch<NotesProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Catatan Kuliah')),
      body: switch (provider.status) {
        NotesStatus.initial ||
        NotesStatus.loading => const Center(child: CircularProgressIndicator()),
        NotesStatus.error => Center(child: Text(provider.errorMessage ?? 'Error')),
        NotesStatus.loaded => Column(
            children: [
              if (provider.isOffline)
                const MaterialBanner(
                  content: Text('Mode offline: menampilkan data cache'),
                  actions: [SizedBox.shrink()],
                ),
              Expanded(
                child: provider.notes.isEmpty
```

```
                    ? const Center(child: Text('Belum ada catatan'))
                    : ListView.builder(
                        itemCount: provider.notes.length,
                        itemBuilder: (context, index) {
                          final note = provider.notes[index];
                          return ListTile(
                            title: Text(note.title),
                            subtitle: Text(note.content),
                          );
                        },
                      ),
                ),
              ],
            ),
          },
        );
      }
    }
```



Fake remote berhasil: daftar catatan tampil



`shouldFail = true` : data cache dan banner mode offline

Screenshot di atas diambil dengan memasang `FakeRemoteDataSource` dan `FakeLocalCache` di `main()`. Ini trik yang berguna untuk mendesain tampilan offline tanpa harus mematikan internet.

Langkah 3: widget test dengan ChangeNotifierProvider

Pola kuncinya: bungkus halaman dengan `ChangeNotifierProvider` yang berisi provider dengan dependensi palsu, **di atas** `MaterialApp`, persis seperti di `main.dart`.

test/widget/notes_page_test.dart

```dart
// test/widget/notes_page_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:catatan_kuliah/data/note_repository.dart';
import 'package:catatan_kuliah/models/note.dart';
import 'package:catatan_kuliah/pages/notes_page.dart';
import 'package:catatan_kuliah/providers/notes_provider.dart';

import '../mock/fakes.dart';

void main() {
  late FakeRemoteDataSource remote;
  late FakeLocalCache cache;

  setUp(() {
    remote = FakeRemoteDataSource(
      awal: [const Note(id: 'n1', title: 'Pertemuan 1', content: 'Dart')],
    );
    cache = FakeLocalCache();
  });

  Widget buatApp() {
    final repository = NoteRepository(remote: remote, cache: cache);
    return ChangeNotifierProvider(
      create: (_) => NotesProvider(repository)..loadNotes(),
      child: const MaterialApp(home: NotesPage()),
    );
  }

  testWidgets('menampilkan loading lalu daftar catatan', (tester) async {
    await tester.pumpWidget(buatApp());

    // Frame pertama: data belum datang
    expect(find.byType(CircularProgressIndicator), findsOneWidget);

    await tester.pumpAndSettle(); // tunggu Future selesai

    expect(find.text('Pertemuan 1'), findsOneWidget);
    expect(find.byType(CircularProgressIndicator), findsNothing);
  });

  testWidgets('offline dengan cache: tampil banner mode offline',
      (tester) async {
    remote.shouldFail = true;
    cache.saved = [const Note(id: 'c1', title: 'Dari cache', content: '-')];

    await tester.pumpWidget(buatApp());
    await tester.pumpAndSettle();

    expect(find.text('Dari cache'), findsOneWidget);
    expect(find.textContaining('Mode offline'), findsOneWidget);
  });

  testWidgets('tanpa data sama sekali: tampil pesan kosong', (tester) async {
    remote.notes.clear();
```

```
    await tester.pumpWidget(buatApp());
    await tester.pumpAndSettle();

    expect(find.text('Belum ada catatan'), findsOneWidget);
  });
}
```

- `NotesProvider(repository)..loadNotes()` : operator kaskade (`..`) memanggil `loadNotes()` lalu mengembalikan provider-nya.
- Frame pertama setelah `pumpWidget` menampilkan loading karena Future belum selesai. `pumpAndSettle()` menunggu Future fake selesai lalu menggambar ulang.
- Fungsi `buatApp()` dipanggil *setelah* test mengubah `remote` / `cache`, sehingga setiap test bisa menyiapkan skenario berbeda.

Alternatif: memakai ChangeNotifierProvider.value

Jika test perlu memegang objek provider untuk dipanggil langsung, buat provider di luar lalu berikan dengan `.value`:

```
testWidgets('catatan baru langsung tampil di daftar', (tester) async {
  final provider = NotesProvider(
    NoteRepository(remote: FakeRemoteDataSource(), cache: FakeLocalCache()),
  );
  await provider.loadNotes();

  await tester.pumpWidget(
    ChangeNotifierProvider.value(
      value: provider,
      child: const MaterialApp(home: NotesPage()),
    ),
  );
  expect(find.text('Belum ada catatan'), findsOneWidget);

  await provider.addNote('Tugas Besar', 'Deadline minggu 16');
  await tester.pump();

  expect(find.text('Tugas Besar'), findsOneWidget);
});
```

⚠️ **Kesalahan umum**

- Membuat provider yang di dalamnya langsung memanggil `Supabase.instance.client` atau `Hive.box()`. Provider seperti itu tidak bisa diuji; terima repository lewat konstruktor.
- Meletakkan `ChangeNotifierProvider` di bawah `MaterialApp` di test, lalu test navigasi gagal dengan *ProviderNotFoundException* (lihat bagian 1).
- Hanya menguji provider lewat UI. Unit test provider jauh lebih cepat dan pesan gagalnya lebih jelas; gunakan widget test untuk memastikan UI bereaksi terhadap state.

10. Golden Test, Integration Test & Coverage

Bagian terakhir membahas tiga alat pelengkap: **golden test** untuk menjaga tampilan tidak berubah tanpa sengaja, **integration test** untuk menguji aplikasi utuh di perangkat, dan **coverage** untuk mengukur seberapa banyak kode yang sudah diuji.

1. Golden test (sekilas)

Golden test merender widget menjadi gambar PNG, lalu membandingkannya dengan gambar acuan (“golden file”) yang disimpan di repo. Jika ada satu pixel yang berbeda, test gagal. Cocok untuk widget yang tampilannya penting dan jarang berubah, seperti kartu atau komponen desain.

`lib/widgets/kartu_nilai.dart`

```dart
// lib/widgets/kartu_nilai.dart
import 'package:flutter/material.dart';

class KartuNilai extends StatelessWidget {
  const KartuNilai({super.key, required this.matkul, required this.nilai});

  final String matkul;
  final String nilai;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        leading: const Icon(Icons.school),
        title: Text(matkul),
        trailing: CircleAvatar(child: Text(nilai)),
      ),
    );
  }
}
```

`test/golden/kartu_nilai_golden_test.dart`

```dart
// test/golden/kartu_nilai_golden_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:catatan_kuliah/widgets/kartu_nilai.dart';

void main() {
  testWidgets('tampilan KartuNilai sesuai golden', (tester) async {
    await tester.pumpWidget(
      const MaterialApp(
        debugShowCheckedModeBanner: false,
        home: Scaffold(
          body: Center(
            child: SizedBox(
              width: 300,
              child: KartuNilai(matkul: 'Pemrograman Mobile', nilai: 'A'),
            ),
          ),
```

```
        ),
      ),
    );

    await expectLater(
      find.byType(KartuNilai),
      matchesGoldenFile('goldens/kartu_nilai.png'),
    );
  });
}
```

```
# Pertama kali: buat gambar acuan (test/golden/goldens/kartu_nilai.png)
flutter test --update-goldens test/golden

# Selanjutnya: bandingkan dengan acuan
flutter test test/golden
```

Jika test golden gagal, Flutter menyimpan gambar perbedaan di folder `test/golden/failures/`. Lihat gambarnya: jika perubahan memang disengaja (misalnya Anda mengganti warna), jalankan lagi `--update-goldens` dan commit gambar barunya.

> ℹ️ **Catatan**
>
> Di lingkungan test, semua teks dirender dengan font khusus berbentuk kotak-kotak (Ahem) agar hasilnya sama di semua komputer. Jadi jangan kaget jika gambar golden tidak menampilkan huruf yang terbaca. Golden file juga bisa sedikit berbeda antara macOS, Windows, dan Linux, sehingga tim biasanya membuat golden di satu OS yang sama (misalnya di CI).

2. Integration test (pengenalan)

Unit dan widget test berjalan di lingkungan virtual. Integration test menjalankan **aplikasi asli** di emulator atau HP, termasuk plugin native (kamera, Hive, Supabase). Gunanya untuk menguji alur penting dari awal sampai akhir, misalnya “register → login → tambah catatan → logout”.

Tambahkan package `integration_test` (bagian dari Flutter SDK) sebagai dev dependency:

```
flutter pub add 'dev:integration_test:{"sdk":"flutter"}'
```

Hasilnya di `pubspec.yaml`:

```
dev_dependencies:
  flutter_test:
    sdk: flutter
  integration_test:
    sdk: flutter
```

Buat folder `integration_test/` di root proyek (sejajar dengan `lib/` dan `test/`):

`integration_test/app_test.dart`

```dart
// integration_test/app_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:catatan_kuliah/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('alur lengkap: buka aplikasi dan tekan tombol', (tester) async {
    app.main(); // jalankan aplikasi ASLI
    await tester.pumpAndSettle();

    expect(find.text('Diklik 0 kali'), findsOneWidget);

    await tester.tap(find.byIcon(Icons.add));
    await tester.pumpAndSettle();

    expect(find.text('Diklik 1 kali'), findsOneWidget);
  });
}
```

```bash
# Pastikan emulator menyala atau HP terhubung
flutter devices

# Jalankan integration test di perangkat
flutter test integration_test/app_test.dart

# Pilih perangkat tertentu
flutter test integration_test -d emulator-5554
```

Perhatikan perbedaannya dengan widget test:

- `IntegrationTestWidgetsFlutterBinding.ensureInitialized()` wajib dipanggil di awal.
- `app.main()` menjalankan fungsi `main()` asli, termasuk inisialisasi Supabase dan Hive. Karena memakai backend sungguhan, gunakan akun dan project Supabase khusus pengujian.
- Integration test jauh lebih lambat (build APK + install), jadi tulis sedikit saja untuk alur yang paling penting.

3. Coverage: seberapa banyak kode yang diuji?

Coverage menghitung persentase baris kode di `lib/` yang dijalankan oleh test.

```bash
# Jalankan semua test sambil merekam coverage
flutter test --coverage

# Hasilnya: coverage/lcov.info (format teks LCOV)
head coverage/lcov.info
```

Isi `lcov.info` berupa daftar file (`SF:`) dan jumlah eksekusi tiap baris (`DA:nomor_baris,jumlah`):

```
SF:lib/utils/ipk.dart
DA:2,2
DA:3,6
DA:4,1
DA:6,1
...
```

Agar mudah dibaca, ubah menjadi laporan HTML dengan `genhtml` dari paket `lcov`:

```
# macOS
brew install lcov
# Ubuntu/Debian
sudo apt install lcov

genhtml coverage/lcov.info -o coverage/html
open coverage/html/index.html        # macOS
xdg-open coverage/html/index.html    # Linux
```

Pengguna Windows atau yang tidak ingin memasang lcov bisa memakai ekstensi VS Code **Coverage Gutters**: ia membaca `coverage/lcov.info` dan mewarnai baris di editor (hijau = teruji, merah = belum).

> 💡 **Tips**
>
> Jangan mengejar 100%. Target realistis untuk tugas besar: logika bisnis (`utils/`, `data/`, `providers/`) di atas 70%, ditambah widget test untuk halaman-halaman utama. Coverage tinggi dengan test yang tidak memeriksa apa-apa (tanpa `expect`) tidak ada artinya.

Ringkasan perintah

Perintah	Fungsi
`flutter analyze`	Cek error dan lint tanpa menjalankan aplikasi
`flutter test`	Jalankan semua unit, widget, dan golden test
`flutter test --update-goldens`	Perbarui gambar acuan golden
`flutter test integration_test`	Jalankan integration test di perangkat
`flutter test --coverage`	Rekam coverage ke `coverage/lcov.info`

> ⚠️ **Kesalahan umum**
>
> - Menaruh file integration test di folder `test/`. File tersebut akan ikut dijalankan oleh `flutter test` biasa dan gagal karena tidak ada perangkat.
> - Lupa meng-commit folder `goldens/`, sehingga test golden gagal di komputer teman satu kelompok.
> - Meng-commit folder `coverage/`. Folder ini hasil generate dan sudah ada di `.gitignore` bawaan Flutter.

Tugas Praktikum: Praktikum 15: Debugging & Suite Pengujian Otomatis Catatan Kuliah

Sasaran Praktikum

Menemukan dan memperbaiki bug dengan alat debugging, lalu menulis rangkaian test otomatis (unit, fake/mock, widget, provider) untuk proyek Catatan Kuliah dari Modul 13-14 sehingga perubahan kode berikutnya tidak merusak fitur lama.

Langkah Pengerjaan

1. **Bagian A - Debugging.** Buat proyek latihan berisi kelima contoh kode "SALAH" dari bagian 1. Jalankan, salin kalimat pertama setiap pesan error dan baris stack trace yang menunjuk ke kode Anda ke dalam file `DEBUGGING.md`, lalu perbaiki satu per satu.
2. Pasang breakpoint di dalam fungsi `getNotes()` pada repository Anda, jalankan dengan F5, dan catat nilai variabel `notes` yang terlihat di panel Variables. Lampirkan screenshot-nya.
3. Buka DevTools tab Network saat halaman catatan dimuat. Catat URL, status code, dan durasi request ke Supabase.
4. Jalankan `flutter analyze` pada proyek Catatan Kuliah, tambahkan aturan `prefer_final_locals` dan `unawaited_futures` ke `analysis_options.yaml`, lalu perbaiki hingga hasilnya `No issues found!`.
5. **Bagian B - Unit test.** Pindahkan validator email/password ke `lib/utils/validators.dart` dan buat `test/unit/validators_test.dart` dengan minimal 6 test (positif, negatif, dan kasus batas).
6. **Bagian C - Fake.** Pastikan `NoteRepository` menerima data source remote dan cache lewat konstruktor. Buat `test/mock/fakes.dart` dan `test/mock/note_repository_test.dart` yang menguji skenario online, offline dengan cache, offline tanpa cache, dan validasi judul kosong.
7. **Bagian D - Provider.** Buat `test/unit/notes_provider_test.dart` yang memeriksa urutan status (loading lalu loaded), penambahan catatan, dan pesan error saat offline.
8. **Bagian E - Widget test.** Buat `test/widget/login_page_test.dart` yang menguji: form kosong, format email salah, password kurang dari 8 karakter, form valid memanggil fungsi login dengan data yang benar, dan SnackBar saat login gagal.
9. Buat `test/widget/notes_page_test.dart` yang memastikan halaman catatan menampilkan loading, daftar catatan, dan banner mode offline.
10. Jalankan `flutter test --coverage`, buat laporan HTML (genhtml atau Coverage Gutters), dan ambil screenshot persentase coverage folder `lib/data` dan `lib/providers`.

Output yang Diharapkan

Repositori berisi file DEBUGGING.md, folder test/ dengan minimal 20 test yang semuanya lulus (output “All tests passed!”), flutter analyze tanpa issue, dan screenshot laporan coverage dengan coverage lib/data dan lib/providers minimal 70%.

Rubrik Penilaian

Laporan debugging lengkap dan perbaikan benar (20%), unit test validator dan provider (20%), test repository dengan fake/mock termasuk skenario gagal (25%), widget test form login dan halaman catatan (25%), flutter analyze bersih dan laporan coverage (10%).

Kuis Pemahaman

1. Saat membaca stack trace error Flutter, baris mana yang paling perlu Anda periksa terlebih dahulu?

A. Baris paling bawah stack trace

B. Baris pertama yang berasal dari package proyek Anda sendiri (bukan package:flutter/)

C. Semua baris yang berasal dari framework.dart

D. Baris yang berisi kata `<asynchronous suspension>`

2. Sebuah ListView.builder diletakkan langsung di dalam Column dan muncul error “Vertical viewport was given unbounded height”. Perbaikan yang paling tepat adalah...

A. Mengganti Column dengan Row

B. Membungkus ListView dengan Expanded agar tingginya mengikuti sisa ruang Column

C. Menambahkan operator ! pada itemCount

D. Memanggil setState setelah ListView dibangun

3. Kode berikut sering memicu error “setState() called after dispose()”: final data = await ambilData(); setState(() ⇒ _data = data);. Apa perbaikannya?

A. Menghapus kata await

B. Menambahkan if (!mounted) return; setelah await dan sebelum setState

C. Mengganti StatefulWidget menjadi StatelessWidget

D. Memanggil dispose() secara manual sebelum setState

4. Mengapa pengukuran performa (mencari jank) di DevTools sebaiknya dilakukan dengan flutter run --profile, bukan mode debug?

A. Karena mode debug tidak bisa membuka DevTools

B. Karena mode debug sengaja lebih lambat akibat banyak pemeriksaan tambahan, sehingga hasil pengukurannya menyesatkan

C. Karena mode profile otomatis memperbaiki jank

D. Karena mode profile tidak membutuhkan perangkat

5. Manakah cara yang BENAR untuk menguji bahwa fungsi biasa hitungIpk melempar ArgumentError?

A. expect(hitungIpk([4], []), throwsArgumentError);

B. expect(() ⇒ hitungIpk([4], []), throwsArgumentError);

C. expect(hitungIpk, ArgumentError);

D. expect(throwsArgumentError, hitungIpk([4], []));

6. Apa tujuan utama membuat FakeRemoteDataSource saat menguji NoteRepository?

A. Agar aplikasi rilis lebih cepat

B. Agar test tidak bergantung pada Supabase/internet dan kondisi seperti offline bisa disimulasikan dengan mudah

C. Agar data di Supabase terhapus setelah test

D. Agar tidak perlu menulis expect

7. Dalam widget test, setelah await tester.tap(find.byIcon(Icons.add)); teks di layar belum berubah. Apa yang terlupa?

A. Memanggil await tester.pump() agar Flutter menggambar frame berikutnya

B. Memanggil setState dari dalam test

C. Menjalankan emulator

D. Menambahkan Key pada Text

8. Perintah apa yang menghasilkan file coverage/lcov.info berisi data baris kode yang dijalankan oleh test?

A. flutter analyze --coverage

B. flutter test --coverage

C. flutter build apk --coverage

D. dart fix --coverage

Kunci Jawaban & Pembahasan

1. B — Stack trace diurutkan dari pemanggilan terakhir. Baris dari package:flutter/ hampir tidak pernah menjadi penyebab; baris pertama dari kode Anda menunjukkan file dan nomor baris tempat masalah dimulai.

2. B — Column memberi tinggi tak terbatas kepada anaknya, sedangkan ListView ingin setinggi mungkin. Expanded memberi ListView batas tinggi berupa sisa ruang Column.

3. B — Selama menunggu Future, pengguna bisa saja meninggalkan halaman sehingga State sudah di-dispose. Properti mounted bernilai false pada kondisi itu, jadi setState tidak boleh dipanggil.

4. B — Mode debug memakai JIT dan banyak assertion sehingga lebih lambat dari aplikasi rilis. Mode profile mendekati performa rilis tetapi tetap bisa diprofil oleh DevTools.

5. B — Fungsi harus dibungkus closure () ⇒ agar expect yang menjalankannya dan bisa menangkap error. Tanpa closure, error terjadi sebelum expect dipanggil sehingga test gagal.

6. B — Fake menggantikan sumber data asli dengan implementasi di memori yang kita kendalikan. Test menjadi cepat, stabil, dan skenario gagal seperti koneksi putus bisa dibuat cukup dengan shouldFail = true.

7. A — tap hanya mengirim sentuhan. Perubahan state baru terlihat setelah frame berikutnya digambar dengan pump() (atau pumpAndSettle() jika ada animasi).

8. B — flutter test --coverage menjalankan semua test sambil merekam baris yang dieksekusi ke coverage/lcov.info, yang kemudian bisa diubah menjadi laporan HTML dengan genhtml atau dibaca ekstensi Coverage Gutters.